Core Java

Interview Question Bank

700 Theory Questions & Answers

1. JDK, JRE, JVM

Q1. What is JVM?

A: JVM (Java Virtual Machine) is an abstract machine that provides a runtime environment to execute Java bytecode. It converts bytecode into machine-specific code and manages memory, garbage collection, and security.

Q2. What is JRE?

A: JRE (Java Runtime Environment) is a software package that provides the libraries, JVM, and other components needed to run Java applications, but it does not include development tools like a compiler.

Q3. What is JDK?

A: JDK (Java Development Kit) is a full software development kit that includes the JRE plus development tools such as the compiler (javac), debugger, and other utilities needed to write and compile Java programs.

Q4. What is the relationship between JDK, JRE, and JVM?

A: JDK contains JRE, and JRE contains JVM. JDK = JRE + development tools; JRE = JVM + libraries. JVM is the core component that actually executes the bytecode.

Q5. Is JVM platform independent?

A: No, JVM itself is platform-dependent; there is a separate JVM implementation for each operating system. However, Java bytecode is platform-independent, which is why Java is described as "write once, run anywhere."

Q6. What is bytecode in Java?

A: Bytecode is the intermediate, platform-independent code generated by the Java compiler (javac) from .java source files. It is stored in .class files and is executed by the JVM.

Q7. What is JIT compiler?

A: JIT (Just-In-Time) compiler is a component of the JVM that improves performance by compiling frequently executed bytecode into native machine code at runtime instead of interpreting it every time.

Q8. What are the main components of JVM?

A: The main components are the Classloader, Runtime Data Areas (method area, heap, stack, PC register, native method stack), Execution Engine (interpreter, JIT compiler, garbage collector), and Native Method Interface.

Q9. What is the Classloader in JVM?

A: The Classloader is a subsystem of JVM responsible for loading class files into memory at runtime. It performs loading, linking, and initialization of classes.

Q10. Why is Java called platform independent if JVM is platform dependent?

A: Java source code is compiled once into bytecode, which is the same across all platforms. Each platform has its own JVM implementation that interprets/executes this same bytecode, so the same .class file can run anywhere a compatible JVM exists.

2. DATA TYPES

Q11. What are data types in Java?

A: Data types specify the type and size of values that can be stored in a variable. Java has two categories: primitive data types (byte, short, int, long, float, double, char, boolean) and non-primitive/reference types (classes, arrays, interfaces).

Q12. How many primitive data types does Java have? List them.

A: Java has 8 primitive data types: byte, short, int, long, float, double, char, and boolean.

Q13. What is the size and range of int in Java?

A: int is 4 bytes (32 bits) and its range is -2,147,483,648 to 2,147,483,647.

Q14. What is the size and range of byte in Java?

A: byte is 1 byte (8 bits) with a range of -128 to 127.

Q15. What is the size and range of short in Java?

A: short is 2 bytes (16 bits) with a range of -32,768 to 32,767.

Q16. What is the size and range of long in Java?

A: long is 8 bytes (64 bits) with a range of approximately -9.2×10^18 to 9.2×10^18. Long literals are suffixed with L.

Q17. What is the size of float and double in Java?

A: float is 4 bytes (single precision) and double is 8 bytes (double precision).

Q18. What is the default value of a float versus a double literal in Java?

A: By default, any decimal literal in Java is treated as double. To assign it to a float variable, you must suffix it with 'f' or 'F', otherwise a compilation error occurs.

Q19. What is char data type in Java?

A: char is a 2-byte (16-bit) unsigned data type used to store a single Unicode character, with a range of 0 to 65,535.

Q20. What is boolean data type in Java?

A: boolean represents one of two values: true or false. Its size is not precisely defined by the JVM specification (typically treated as 1 bit conceptually, though JVM implementation may vary).

Q21. What are non-primitive (reference) data types in Java?

A: Non-primitive data types include classes, interfaces, arrays, and Strings. They store references (memory addresses) to objects rather than the actual values.

Q22. What is the difference between primitive and reference data types?

A: Primitive types store actual values directly in memory (stack), have fixed sizes, and are predefined by the language. Reference types store the address of an object located in heap memory, and their behavior is defined by classes.

Q23. What are default values of primitive data types in Java?

A: byte, short, int → 0; long → 0L; float → 0.0f; double → 0.0d; char → '\u0000'; boolean → false. These defaults apply to instance and static variables, not local variables.

Q24. Do local variables get default values in Java?

A: No, local variables must be explicitly initialized before use; Java does not assign them default values, and the compiler throws an error if used uninitialized.

Q25. What is type conversion in the context of data types?

A: Type conversion is the process of converting one data type into another, either automatically (implicit/widening) or manually (explicit/narrowing) by the programmer.

Q26. Why is char in Java 2 bytes while it is 1 byte in C/C++?

A: Java uses Unicode to support internationalization, which requires 2 bytes to represent a wide range of characters from different languages, unlike C/C++ which typically uses ASCII (1 byte).

Q27. Can you store a negative value in a char variable?

A: No, char is an unsigned data type in Java and can only store non-negative values (0 to 65535).

Q28. What happens if you assign an integer literal larger than the int range without a suffix?

A: It causes a compile-time error since Java assumes int by default for whole number literals; you must add an 'L' suffix to indicate a long literal.

Q29. What is the difference between float and double in terms of precision?

A: float provides about 6-7 decimal digits of precision (single precision, 32-bit), while double provides about 15-16 decimal digits of precision (double precision, 64-bit), making double more accurate for calculations.

Q30. Why does Java not have unsigned data types (except char)?

A: Java's designers chose to omit unsigned integer types to simplify the language and reduce programmer errors related to sign conversions; char remains the exception because it represents characters, not numeric quantities.

3. OPERATORS

Q31. What are operators in Java?

A: Operators are special symbols that perform operations on variables and values (operands), such as arithmetic, relational, logical, bitwise, and assignment operations.

Q32. What are the types of operators in Java?

A: Arithmetic, unary, assignment, relational (comparison), logical, bitwise, shift, ternary (conditional), and instanceof operators.

Q33. What are arithmetic operators in Java?

A: +, -, *, /, and % (modulus), used to perform basic mathematical operations.

Q34. What is the modulus (%) operator used for?

A: It returns the remainder after division of two operands, e.g., 10 % 3 equals 1.

Q35. What are relational operators in Java?

A: ==, !=, >, <, >=, and <=, used to compare two values and return a boolean result.

Q36. What are logical operators in Java?

A: && (logical AND), || (logical OR), and ! (logical NOT), used to combine or invert boolean expressions.

Q37. What is the difference between & and && in Java?

A: & is a bitwise/logical AND that evaluates both operands regardless of the first one's result, while && is a short-circuit logical AND that skips evaluating the second operand if the first is false.

Q38. What is short-circuit evaluation?

A: It is behavior where the second operand of && or || is not evaluated if the result can already be determined from the first operand (e.g., in ||, if the first is true, the second is skipped).

Q39. What are bitwise operators in Java?

A: &, |, ^ (XOR), and ~ (bitwise complement), which operate on individual bits of integer types.

Q40. What are shift operators in Java?

A: << (left shift), >> (signed right shift), and >>> (unsigned right shift), used to shift bits of a number left or right.

Q41. What is the difference between >> and >>>?

A: >> is a signed right shift that preserves the sign bit (fills with the sign bit on the left), whereas >>> is an unsigned right shift that always fills with 0 regardless of sign.

Q42. What is the ternary operator in Java?

A: It is a shorthand conditional operator written as condition ? expr1 : expr2, which evaluates expr1 if the condition is true and expr2 otherwise.

Q43. What is the instanceof operator?

A: It tests whether an object is an instance of a specific class or interface, returning a boolean result, e.g., obj instanceof String.

Q44. What are unary operators in Java?

A: Operators that work on a single operand: + (unary plus), - (unary minus), ++ (increment), -- (decrement), and ! (logical complement).

Q45. What is the difference between pre-increment and post-increment?

A: Pre-increment (++x) increments the value first and then uses it in the expression, while post-increment (x++) uses the current value in the expression first and then increments it.

Q46. What are assignment operators in Java?

A: =, and compound assignment operators like +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=.

Q47. What is operator precedence?

A: It is the order in which operators are evaluated in an expression when multiple operators are present; higher-precedence operators are evaluated before lower-precedence ones.

Q48. What is operator associativity?

A: It determines the order in which operators of the same precedence are evaluated—either left-to-right or right-to-left.

Q49. Can the + operator be used with Strings?

A: Yes, the + operator is overloaded for String concatenation in Java, joining two strings or a string with another data type together.

Q50. What is the result of 5/2 and 5.0/2 in Java?

A: 5/2 results in 2 (integer division, truncates decimal), while 5.0/2 results in 2.5 (floating-point division).

4. BASICS OF CLASSES AND OBJECTS

Q51. What is a class in Java?

A: A class is a blueprint or template that defines the properties (fields) and behaviors (methods) that objects created from it will have.

Q52. What is an object in Java?

A: An object is an instance of a class, representing a real-world entity with state (field values) and behavior (methods), created using the 'new' keyword.

Q53. What is the difference between a class and an object?

A: A class is a logical/conceptual template with no memory allocated for data, while an object is a concrete instance of that class with memory allocated at runtime.

Q54. How do you create an object in Java?

A: Using the 'new' keyword followed by a constructor call, e.g., ClassName obj = new ClassName();

Q55. What is state and behavior of an object?

A: State refers to the values held in an object's fields/attributes at a given time; behavior refers to the actions or methods the object can perform.

Q56. What is instantiation in Java?

A: Instantiation is the process of creating an object (an instance) from a class using the new keyword.

Q57. Where are objects stored in memory?

A: Objects are stored in heap memory, while references to those objects are typically stored in stack memory (for local variables).

Q58. What is a constructor's role in object creation?

A: A constructor initializes the newly created object's state by assigning initial values to its fields when the object is instantiated.

Q59. Can a class exist without any objects?

A: Yes, a class can be defined and compiled without ever being instantiated into an object; it will simply not be used to create runtime instances.

Q60. What is the difference between an instance variable and a local variable?

A: Instance variables are declared inside a class but outside any method and belong to an object (each object gets its own copy), while local variables are declared inside a method/block and exist only during that method's execution.

Q61. What does 'new' keyword do internally?

A: It allocates memory on the heap for a new object, invokes the appropriate constructor to initialize the object, and returns a reference to that memory location.

Q62. Can a Java file contain multiple classes?

A: Yes, a single .java file can contain multiple classes, but at most one public class, and the public class name must match the file name.

Q63. What is the difference between an object reference and the object itself?

A: The object reference is a variable holding the memory address of the object, while the object itself is the actual data structure stored in heap memory; multiple references can point to the same object.

Q64. What happens when you assign one object reference to another?

A: Both references point to the same object in memory; no new object is created, so changes made through one reference are visible through the other.

5. NAMING CONVENTIONS IN JAVA

Q65. What are naming conventions in Java?

A: Naming conventions are established rules for naming identifiers (classes, methods, variables, constants, packages) to improve code readability and maintainability, though not enforced by the compiler.

Q66. What is the naming convention for classes and interfaces?

A: Class and interface names should start with an uppercase letter and follow CamelCase, e.g., Employee, ArrayList.

Q67. What is the naming convention for methods and variables?

A: Methods and variables should start with a lowercase letter and follow camelCase, e.g., calculateSalary(), employeeName.

Q68. What is the naming convention for constants?

A: Constants (typically declared as static final) should be written in all uppercase letters with underscores separating words, e.g., MAX_VALUE.

Q69. What is the naming convention for packages?

A: Package names should be all lowercase, often using reversed domain names for uniqueness, e.g., com.company.project.

Q70. Are naming conventions enforced by the Java compiler?

A: No, naming conventions are recommended style guidelines, not compiler-enforced rules; code that violates them will still compile and run.

Q71. Can a Java identifier start with a digit?

A: No, identifiers must start with a letter, underscore (_), or dollar sign ($); they cannot begin with a digit.

Q72. Can Java identifiers contain special characters?

A: Java identifiers can only contain letters, digits, underscores, and dollar signs—no other special characters or spaces are allowed.

Q73. Why is following naming conventions important even though not mandatory?

A: It improves code readability, consistency across teams, easier maintenance, and helps other developers quickly understand the purpose of identifiers.

6. RESERVED WORDS (KEYWORDS)

Q74. What are reserved words/keywords in Java?

A: Reserved words are predefined words that have special meaning in the Java language and cannot be used as identifiers (variable, method, or class names).

Q75. How many keywords does Java have?

A: Java has 53 reserved keywords (including 2 reserved for future use: 'const' and 'goto', which are not currently functional).

Q76. Are 'true', 'false', and 'null' keywords in Java?

A: No, 'true', 'false', and 'null' are literals, not keywords, though they are still reserved and cannot be used as identifiers.

Q77. What are 'const' and 'goto' in Java?

A: They are reserved keywords that exist in the language but are not implemented/used; they were reserved to prevent conflicts with C/C++ style code and possible future use.

Q78. What is the purpose of access modifier keywords (public, private, protected)?

A: They control the visibility/accessibility of classes, methods, and variables to other classes and packages.

Q79. What do keywords like 'class', 'interface', 'extends', and 'implements' represent?

A: 'class' and 'interface' declare types; 'extends' is used for inheritance between classes (or interfaces); 'implements' is used by a class to implement an interface.

Q80. What are control flow keywords in Java?

A: if, else, switch, case, default, for, while, do, break, continue, return.

Q81. What are exception handling keywords in Java?

A: try, catch, finally, throw, throws.

Q82. What are keywords related to data types?

A: byte, short, int, long, float, double, char, boolean.

Q83. What is the 'this' keyword used for as a reserved word?

A: It refers to the current instance of the class, used to distinguish instance variables from parameters/local variables, or to call other constructors.

Q84. What is the 'super' keyword used for?

A: It refers to the immediate parent class object, used to access parent class members or call the parent constructor.

Q85. What does the 'static' keyword signify?

A: It indicates that a member (variable, method, block, or nested class) belongs to the class itself rather than any particular instance.

Q86. What does 'final' keyword mean when applied to variables, methods, and classes?

A: For a variable, it makes the value constant (unmodifiable after initialization); for a method, it prevents overriding; for a class, it prevents inheritance (extension).

Q87. What is the 'abstract' keyword used for?

A: It is used to declare abstract classes (which cannot be instantiated) and abstract methods (which have no body and must be implemented by subclasses).

Q88. What does 'synchronized' keyword do?

A: It is used to control access to a method or block by multiple threads, ensuring only one thread can execute it at a time on a given object/class lock.

Q89. What is 'volatile' keyword used for?

A: It indicates that a variable's value may be modified by multiple threads, ensuring that reads/writes go directly to main memory rather than thread-local caches.

Q90. What is 'transient' keyword used for?

A: It marks a field to be excluded from the default serialization process of an object.

Q91. What does 'native' keyword indicate?

A: It indicates that a method is implemented in platform-specific code (typically C/C++) outside of Java, via JNI (Java Native Interface).

Q92. What is the 'instanceof' keyword?

A: It is an operator/keyword used to test whether an object is an instance of a particular class or interface.

Q93. What are 'package' and 'import' keywords used for?

A: 'package' declares the namespace a class belongs to; 'import' brings in classes/interfaces from other packages for use in the current file.

7. METHODS AND THEIR TYPES

Q94. What is a method in Java?

A: A method is a block of code that performs a specific task, has a name, may accept parameters, and may return a value; it is defined within a class.

Q95. What is the syntax/structure of a method?

A: accessModifier returnType methodName(parameterList) { method body }, e.g., public int add(int a, int b) { return a+b; }

Q96. What are the types of methods in Java?

A: Predefined (built-in library) methods and user-defined methods; also categorized as static methods and instance (non-static) methods.

Q97. What is the difference between predefined and user-defined methods?

A: Predefined methods are already implemented in Java's standard library (e.g., length(), print()), while user-defined methods are written by the programmer for specific application needs.

Q98. What is a method signature?

A: The method signature consists of the method name and the parameter list (types, order, and number), excluding the return type and access modifier.

Q99. What is the difference between a method and a function?

A: In Java, all functions are called methods because they are always associated with a class or object; Java does not support standalone functions outside of classes.

Q100. What is a void method?

A: A method that does not return any value; its return type is declared as 'void'.

Q101. Can a method return multiple values in Java?

A: Not directly; a method can return only one value, but you can return multiple values indirectly using an array, collection, or a custom object/class.

Q102. What is a parameter versus an argument?

A: A parameter is the variable defined in the method signature that receives a value, while an argument is the actual value passed to the method when it is called.

Q103. What is method invocation/calling?

A: It is the process of executing a method by referencing its name along with required arguments, e.g., obj.methodName(args).

Q104. What is pass by value in Java?

A: Java is strictly pass-by-value; when a method is called, a copy of the argument's value (or reference value for objects) is passed, so reassigning the parameter inside the method does not affect the original variable.

Q105. If Java is pass-by-value, why do changes to an object's fields inside a method reflect outside?

A: Because the value passed is a copy of the reference (memory address); both the original and copied reference point to the same object, so modifying the object's internal state through either reference affects the same object.

Q106. What is a recursive method?

A: A method that calls itself directly or indirectly to solve a problem by breaking it into smaller subproblems, typically requiring a base case to terminate.

Q107. What is method accessibility/scope?

A: It is determined by access modifiers (public, private, protected, default) which control which classes can call the method.

Q108. Can a method have no parameters?

A: Yes, a method can have zero parameters, e.g., public void display() { }.

Q109. What is a getter and setter method?

A: Getter methods return the value of a private field (e.g., getName()), and setter methods update the value of a private field (e.g., setName(String name)), used to encapsulate data.

Q110. Why do we use getters and setters instead of public fields?

A: They provide controlled access to private fields, allowing validation logic, read-only or write-only access, and better encapsulation while keeping internal implementation hidden.

8. METHOD OVERLOADING

Q111. What is method overloading in Java?

A: Method overloading is defining multiple methods in the same class with the same name but different parameter lists (different number, type, or order of parameters).

Q112. What are the ways to achieve method overloading?

A: By changing the number of parameters, changing the data types of parameters, or changing the order of parameter types.

Q113. Can methods be overloaded by changing only the return type?

A: No, changing only the return type while keeping the same method name and parameter list causes a compile-time error, since the return type alone does not differentiate method signatures.

Q114. Can methods be overloaded by changing only the access modifier?

A: No, access modifiers do not contribute to the method signature, so changing only the access modifier does not achieve overloading and will cause a compilation error if parameters remain identical.

Q115. What is compile-time polymorphism? How does it relate to overloading?

A: Compile-time (static) polymorphism refers to a behavior resolved at compile time based on method signatures; method overloading is a classic example since the compiler decides which overloaded method to call based on argument types at compile time.

Q116. Can constructors be overloaded?

A: Yes, constructor overloading is common—defining multiple constructors with different parameter lists in the same class.

Q117. What happens if there is ambiguity in overloaded method calls (e.g., passing null)?

A: If the compiler cannot determine a single best-matching overloaded method (e.g., ambiguous type matching), it results in a compile-time "reference to method is ambiguous" error.

Q118. How does Java resolve which overloaded method to call when widening is possible?

A: Java prefers exact type matches first; if none exist, it looks for a match via automatic widening (e.g., int to long); if none, it tries autoboxing; and finally varargs, in that order of preference.

Q119. Can you overload the main() method in Java?

A: Yes, main() can be overloaded with different parameter lists, but JVM will only invoke the standard public static void main(String[] args) as the program entry point; other overloads must be called explicitly.

Q120. What is the benefit of method overloading?

A: It improves code readability and reusability by allowing the same method name to handle different types/numbers of inputs, avoiding the need for differently named methods for similar operations.

9. TYPE CASTING

Q121. What is type casting in Java?

A: Type casting is the process of converting a variable from one data type to another, either implicitly (widening) or explicitly (narrowing).

Q122. What is widening (implicit) type casting?

A: It is the automatic conversion of a smaller data type to a larger data type (e.g., int to long, float to double) performed by the compiler without data loss, since the target type can hold all values of the source type.

Q123. What is narrowing (explicit) type casting?

A: It is the manual conversion of a larger data type to a smaller data type (e.g., double to int), which must be explicitly specified using a cast operator because it may result in data loss.

Q124. What is the order of widening for primitive types?

A: byte → short → int → long → float → double (char also converts to int and beyond). This is the sequence in which smaller types can be automatically widened to larger ones.

Q125. How do you perform explicit casting in Java? Give an example.

A: By placing the target type in parentheses before the value, e.g., int x = (int) 9.7; which converts 9.7 (double) to 9 (int), truncating the decimal part.

Q126. What happens when you cast a double to an int?

A: The decimal/fractional part is truncated (not rounded), e.g., (int) 9.99 results in 9.

Q127. What is the difference between implicit and explicit casting in terms of data loss?

A: Implicit (widening) casting is safe and does not lose data since it moves to a type with greater range/precision; explicit (narrowing) casting can lose data or precision since it moves to a type with a smaller range.

Q128. Can you cast a boolean to an int in Java?

A: No, Java does not allow casting between boolean and numeric types; boolean is not compatible with any numeric type conversion.

Q129. What is casting between object types (upcasting and downcasting)?

A: Upcasting is converting a subclass reference to a superclass type (implicit, always safe); downcasting is converting a superclass reference back to a subclass type (explicit, requires a cast and can throw ClassCastException if invalid).

Q130. What is ClassCastException?

A: It is a runtime exception thrown when an attempt is made to cast an object to a subclass/type of which it is not actually an instance.

Q131. How can you safely perform downcasting to avoid ClassCastException?

A: By using the instanceof operator to check the actual object type before performing the downcast.

Q132. What happens when you assign a byte value that exceeds the int range in a mixed expression?

A: In expressions, byte, short, and char are automatically promoted to int, so there is no issue unless the final result needs to be stored back into a smaller type, which then requires explicit narrowing casting.

10. SCANNER

Q133. What is the Scanner class in Java?

A: Scanner is a class in java.util package used to read input from various sources such as the console, files, or strings, parsing primitive types and strings using regular expressions.

Q134. How do you create a Scanner object to read console input?

A: Scanner sc = new Scanner(System.in);

Q135. What package does the Scanner class belong to?

A: java.util package, so it must be imported using import java.util.Scanner;

Q136. What are common methods of the Scanner class?

A: nextInt(), nextLong(), nextDouble(), nextFloat(), next() (reads a word/token), nextLine() (reads a full line), nextBoolean(), and hasNext() variants to check availability of input.

Q137. What is the difference between next() and nextLine()?

A: next() reads input only until the next whitespace (a single token/word), whereas nextLine() reads the entire line including spaces until a newline character.

Q138. Why does calling nextInt() followed by nextLine() sometimes cause issues?

A: nextInt() reads only the numeric token and leaves the trailing newline character in the input buffer; the subsequent nextLine() call then reads that leftover newline as an empty string instead of waiting for new input.

Q139. How do you fix the nextInt()/nextLine() buffer issue?

A: By adding an extra sc.nextLine() call immediately after nextInt() to consume the leftover newline character before reading the next full line.

Q140. What does hasNextInt() do?

A: It checks whether the next token in the input can be interpreted as an int, returning true or false without consuming the input, often used to validate input before reading it.

Q141. Is it necessary to close a Scanner object? Why?

A: It is good practice to close it using sc.close() to release the underlying resources, although closing a Scanner wrapping System.in also closes System.in, which can prevent further console input in that program.

Q142. Can Scanner read input from a file?

A: Yes, by passing a File object to the constructor, e.g., Scanner sc = new Scanner(new File("data.txt"));, which requires handling FileNotFoundException.

Q143. What exception does Scanner throw if the input format doesn't match the expected type?

A: It throws InputMismatchException, e.g., if nextInt() is called but the input is a non-numeric string.

11. ARRAYS IN JAVA

Q144. What is an array in Java?

A: An array is a container object that holds a fixed number of values of a single data type, stored in contiguous memory locations and accessed via a zero-based index.

Q145. How do you declare and initialize an array in Java?

A: int[] arr = new int[5]; (declaration with size) or int[] arr = {1,2,3,4,5}; (declaration with initialization).

Q146. Are arrays in Java objects?

A: Yes, arrays are objects in Java, created on the heap, and they have a fixed 'length' property (not a method).

Q147. What is the default value of array elements?

A: It depends on the data type: numeric types default to 0, boolean defaults to false, char defaults to '\u0000', and reference types (objects) default to null.

Q148. What is the index range of an array of size n?

A: The valid indices range from 0 to n-1.

Q149. What exception is thrown when accessing an invalid array index?

A: ArrayIndexOutOfBoundsException, a runtime exception thrown when accessing an index less than 0 or greater than or equal to the array's length.

Q150. Can array size be changed after creation?

A: No, arrays in Java are of fixed size once created; to "resize," a new array must be created and the elements copied over (e.g., using Arrays.copyOf()).

Q151. What is a multidimensional array?

A: An array of arrays, such as a 2D array (int[][] arr), used to represent matrix-like or tabular data structures.

Q152. How do you declare a 2D array in Java?

A: int[][] arr = new int[3][4]; creates a 2D array with 3 rows and 4 columns, or int[][] arr = {{1,2},{3,4}}; using initializer syntax.

Q153. Can a 2D array in Java have rows of different lengths (jagged array)?

A: Yes, Java supports jagged arrays where each row can have a different number of columns, since a 2D array is actually an array of array references.

Q154. How do you find the length of a 2D array?

A: arr.length gives the number of rows, and arr[i].length gives the number of columns in row i.

Q155. What is the difference between array length (property) and String length() (method)?

A: Arrays use 'length' as a public final field (no parentheses, e.g., arr.length), whereas String uses length() as a method (with parentheses, e.g., str.length()).

Q156. How do you copy an array in Java?

A: Using System.arraycopy(), Arrays.copyOf(), Arrays.copyOfRange(), clone(), or manually iterating and copying elements.

Q157. What does the clone() method do on an array?

A: It creates a shallow copy of the array—a new array with the same elements; for arrays of primitives, this copies actual values, but for arrays of objects, only references are copied (not deep-cloned).

Q158. How do you sort an array in Java?

A: Using Arrays.sort(arrayName), which sorts primitive arrays using a dual-pivot quicksort and object arrays using a modified mergesort (Timsort), in ascending order by default.

Q159. How do you search for an element in a sorted array efficiently?

A: Using Arrays.binarySearch(array, key), which performs a binary search in O(log n) time, but requires the array to be sorted beforehand.

Q160. What is an array of objects?

A: An array where each element is a reference to an object of a class, e.g., Employee[] emp = new Employee[5]; each element must be individually instantiated before use, otherwise it holds null.

Q161. Can you store elements of different types in a single array?

A: Not directly in a normal typed array, but you can create an array of type Object[] to store mixed types since all classes inherit from Object; however, this requires casting when retrieving elements.

Q162. How is memory allocated for arrays in Java?

A: Arrays are allocated on the heap; the reference variable pointing to the array is stored on the stack (if local) while the actual array data resides in heap memory.

Q163. What is the difference between int[] arr and int arr[] in declaration?

A: Both are valid syntaxes in Java for declaring an array; int[] arr is the preferred/conventional style, while int arr[] is a C-style syntax also supported for compatibility.

Q164. Can array size be negative or zero?

A: Array size cannot be negative (throws NegativeArraySizeException at runtime), but it can be zero, creating a valid array with no elements.

12. STRINGS IN JAVA

Q165. What is a String in Java?

A: A String is an object representing a sequence of characters; it is implemented as a final class in the java.lang package and is immutable.

Q166. Why are Strings immutable in Java?

A: For security (used in class loading, network connections), thread-safety (shared safely without synchronization), caching of hashcode, and to support the String pool optimization for memory efficiency.

Q167. What is the String pool (String constant pool)?

A: It is a special memory area within the heap where Java stores unique String literals; when a new String literal is created, Java checks the pool first and reuses an existing object if the value already exists, saving memory.

Q168. What is the difference between creating a String using a literal versus the 'new' keyword?

A: A String literal (e.g., "hello") is placed in/reused from the String pool, while new String("hello") always creates a new object in heap memory outside the pool (though its internal char data may still reference pool content).

Q169. What does the intern() method do?

A: It returns a reference to the String pool version of the string's value, adding it to the pool if not already present, allowing manually-created String objects to be pooled.

Q170. How do you compare two Strings for content equality?

A: Using the equals() method (or equalsIgnoreCase() for case-insensitive comparison), which compares actual character content rather than reference addresses.

Q171. What is the difference between == and .equals() for Strings?

A: == compares reference (memory address) equality, checking if two variables point to the same object, while .equals() compares the actual character content/value of the strings.

Q172. What is StringBuffer in Java?

A: StringBuffer is a mutable sequence of characters, similar to String but modifiable in place; it is thread-safe because its methods are synchronized.

Q173. What is StringBuilder in Java?

A: StringBuilder is a mutable sequence of characters like StringBuffer, but it is not thread-safe (not synchronized), making it faster for single-threaded use.

Q174. What is the difference between String, StringBuffer, and StringBuilder?

A: String is immutable; StringBuffer is mutable and synchronized (thread-safe but slower); StringBuilder is mutable and unsynchronized (not thread-safe but faster), typically preferred in single-threaded contexts.

Q175. What are common String methods?

A: length(), charAt(), substring(), indexOf(), lastIndexOf(), replace(), trim(), toUpperCase(), toLowerCase(), split(), concat(), contains(), startsWith(), endsWith(), isEmpty(), and isBlank().

Q176. What does the substring() method do?

A: It returns a new string that is a portion of the original string, specified by start index (inclusive) and optionally an end index (exclusive).

Q177. What is the difference between isEmpty() and isBlank()?

A: isEmpty() returns true only if the string has zero length; isBlank() (Java 11+) returns true if the string is empty or contains only whitespace characters.

Q178. How do you convert a String to a character array?

A: Using the toCharArray() method, which returns a char[] containing all characters of the string.

Q179. How do you convert a String to an int and vice versa?

A: Integer.parseInt(str) converts a String to int; String.valueOf(intValue) or Integer.toString(intValue) converts an int to String.

Q180. What happens internally when you concatenate Strings using the + operator in a loop?

A: Each concatenation creates a new String object (since Strings are immutable), which is inefficient in loops; the compiler often optimizes single-line concatenations using StringBuilder internally, but explicit loops do not get this optimization automatically.

Q181. Why is StringBuilder recommended over String for concatenation in loops?

A: Because StringBuilder modifies its internal character array in place without creating new objects each time, making it significantly more memory and performance efficient than repeated String concatenation.

Q182. What is the difference between String.equals() and String.compareTo()?

A: equals() returns a boolean indicating whether two strings have identical content, while compareTo() returns an int indicating lexicographic order (0 if equal, negative if the calling string is "less than," positive if "greater than").

Q183. How do you reverse a String in Java?

A: Commonly by using new StringBuilder(str).reverse().toString(), since String itself has no built-in reverse() method.

Q184. What does the split() method do?

A: It divides a string into an array of substrings based on a specified regular expression delimiter.

Q185. Is String thread-safe? Why?

A: Yes, String is inherently thread-safe because it is immutable—once created, its state cannot change, so multiple threads can safely share the same String object without synchronization.

Q186. What is the hashCode() and equals() contract for Strings?

A: Two equal Strings (per equals()) must return the same hashCode(); String overrides both methods to base them on character content, ensuring consistent behavior in hash-based collections like HashMap and HashSet.

Q187. How many objects are created by String s = new String("hello")?

A: Potentially two: one in the String pool (if "hello" doesn't already exist there, created due to the literal) and one in the heap (created explicitly by 'new'), with 's' referencing the heap object.

Q188. What is string formatting in Java? Give an example.

A: Using String.format() or printf() to create formatted strings with placeholders, e.g., String.format("Name: %s, Age: %d", name, age).

13. THIS KEYWORD

Q189. What is the 'this' keyword in Java?

A: 'this' is a reference variable that refers to the current object—the instance on which a method or constructor is currently being invoked.

Q190. What are the uses of the 'this' keyword?

A: To refer to current class instance variables (resolving naming conflicts), to invoke current class methods, to invoke current class constructors (this()), to pass the current object as a parameter, and to return the current object from a method.

Q191. Can 'this' be used in a static context?

A: No, 'this' cannot be used inside static methods or static blocks because it refers to an instance, and static members belong to the class rather than any specific object.

Q192. How does 'this' help resolve variable shadowing?

A: When a parameter or local variable has the same name as an instance variable, 'this.variableName' explicitly refers to the instance variable, distinguishing it from the local one.

Q193. What is this() used for (with parentheses)?

A: this() is used to call another constructor of the same class (constructor chaining), and must be the first statement in the calling constructor.

Q194. Can this() and super() both be used in the same constructor?

A: No, only one of this() or super() can be used in a constructor, and it must be the first statement, since both cannot simultaneously be the first line.

Q195. Can 'this' be passed as an argument to a method?

A: Yes, 'this' can be passed to another method or constructor to give it a reference to the current object, useful in scenarios like listener registration or builder patterns.

14. SHADOWING (NAME CLASH BETWEEN LOCAL AND INSTANCE VARIABLES)

Q196. What is variable shadowing in Java?

A: Shadowing occurs when a local variable or parameter has the same name as an instance variable, causing the local variable to "shadow" (hide) the instance variable within that scope.

Q197. How do you access a shadowed instance variable inside a method?

A: By prefixing it with 'this', e.g., this.variableName, to explicitly reference the instance variable rather than the local one.

Q198. Does shadowing cause a compilation error in Java?

A: No, shadowing is legal in Java and does not cause a compile error; however, it can cause confusion and subtle bugs if not handled carefully using 'this'.

Q199. Can a local variable shadow a static variable of the same class?

A: Yes, a local variable can shadow a static variable too; in that case, ClassName.variableName or just the static variable accessed via the class name can be used to refer to the static one explicitly.

Q200. What is the difference between shadowing and overriding?

A: Shadowing applies to variables (hiding a variable with the same name in a narrower scope) and is resolved at compile time based on reference type, whereas overriding applies to instance methods and is resolved at runtime based on the actual object type (dynamic/runtime polymorphism).

Q201. Give an example scenario where shadowing can cause a bug.

A: In a constructor like public Employee(String name) { name = name; }, without 'this', the parameter reassigns itself instead of updating the instance variable, leaving the instance field with its default value (null).

15. CONSTRUCTORS AND THEIR PROPERTIES

Q202. What is a constructor in Java?

A: A constructor is a special block of code, resembling a method, that is automatically invoked when an object is instantiated, used to initialize the object's state.

Q203. What are the properties/rules of a constructor?

A: A constructor must have the same name as the class, has no return type (not even void), can be overloaded, cannot be abstract, static, final, or synchronized, and is invoked automatically via the 'new' keyword.

Q204. Why doesn't a constructor have a return type?

A: Because a constructor's purpose is solely to initialize an object, not to return a value; implicitly, it "returns" the newly created object reference, but this is handled by the JVM, not declared explicitly.

Q205. Can a constructor be inherited?

A: No, constructors are not inherited by subclasses because they are tied to the class name; however, a subclass constructor can invoke a parent constructor using super().

Q206. Can constructors be private? What is the use case?

A: Yes, constructors can be private, commonly used in Singleton design patterns to prevent external instantiation, forcing object creation through a static factory method instead.

Q207. Can a constructor call another method of the class?

A: Yes, a constructor can call other instance or static methods of the class as part of its initialization logic.

Q208. What is constructor chaining?

A: It is the process of calling one constructor from another within the same class (using this()) or from a parent class (using super()), allowing shared initialization logic to be reused.

Q209. Is a constructor a method?

A: No, technically a constructor is not a method—it doesn't have a return type, cannot be inherited, and cannot be called directly like a normal method (only invoked via 'new' or chaining).

Q210. What happens if you don't explicitly define any constructor in a class?

A: The Java compiler automatically provides a no-argument default constructor that calls the superclass's no-arg constructor implicitly.

16. DEFAULT AND PARAMETERIZED CONSTRUCTORS

Q211. What is a default constructor?

A: A no-argument constructor automatically generated by the Java compiler if no constructor is explicitly defined in a class; it initializes fields to their default values and calls super().

Q212. What is a parameterized constructor?

A: A constructor that accepts one or more parameters, allowing the object's fields to be initialized with specific values at the time of creation.

Q213. If a class has a parameterized constructor, will Java still provide a default constructor?

A: No, if any constructor (parameterized or otherwise) is explicitly defined, the compiler does not generate a default no-arg constructor; if a no-arg constructor is needed, it must also be explicitly written.

Q214. What is the difference between a default constructor and a "no-argument constructor written by the programmer"?

A: A default constructor is auto-generated by the compiler only when no constructors exist; a no-arg constructor written explicitly by the programmer is technically not called a "default constructor" even though it behaves similarly, since the term specifically refers to compiler-generated ones.

Q215. Can you have both default (no-arg) and parameterized constructors in the same class?

A: Yes, through constructor overloading, a class can have multiple constructors with different parameter lists, including one with no parameters.

Q216. What values does the compiler-generated default constructor assign to fields?

A: It assigns the standard default values based on data type (0 for numeric types, false for boolean, null for objects, '\u0000' for char).

17. STATIC KEYWORD

Q217. What is the static keyword in Java?

A: static is a keyword used to indicate that a member (variable, method, block, or nested class) belongs to the class itself rather than to any specific instance, shared across all objects of that class.

Q218. What can be declared static in Java?

A: Variables, methods, blocks, and nested (inner) classes can be declared static.

Q219. Why do we use the static keyword?

A: To create members that are shared across all instances (memory efficient for common data), to allow access without creating an object, and to define utility/helper methods that don't depend on instance state.

18. STATIC VARIABLES

Q220. What is a static variable?

A: A static variable (also called a class variable) is a variable declared with the static keyword; it is shared among all instances of the class and stored in a single memory location regardless of how many objects are created.

Q221. How is a static variable different from an instance variable?

A: A static variable belongs to the class and has a single copy shared by all objects, while an instance variable belongs to each object individually, with each object maintaining its own separate copy.

Q222. When is memory allocated for a static variable?

A: Memory for static variables is allocated once, at class loading time, and persists for the lifetime of the application (stored in the method area/metaspace).

Q223. Can static variables be accessed without creating an object?

A: Yes, static variables can be accessed directly using the class name (ClassName.variableName), without needing to instantiate the class.

Q224. Give a practical example of when to use a static variable.

A: A counter variable to track the number of objects created from a class, since it needs to be shared and updated across all instances (e.g., a static 'count' variable incremented in the constructor).

Q225. Can a static variable be changed via an instance reference?

A: Yes, syntactically you can access/modify it via obj.variableName, but since there's only one copy, the change is reflected for all objects/references (this usage is discouraged; class name access is preferred).

19. STATIC BLOCKS

Q226. What is a static block in Java?

A: A static block is a block of code prefixed with the static keyword, used to initialize static variables or perform one-time setup activities when the class is loaded into memory.

Q227. When does a static block execute?

A: It executes once, when the class is first loaded by the JVM (classloading), before any static methods are called or objects are created, and before the main method runs (if in the same class).

Q228. Can a class have multiple static blocks?

A: Yes, a class can have multiple static blocks, and they execute in the order they appear in the source code, from top to bottom.

Q229. What is the execution order between static blocks, instance blocks, and constructors?

A: Static blocks run first (once, at class loading), then for each object creation: instance blocks run, followed by the constructor.

Q230. What is a common use case for static blocks?

A: Initializing static variables that require complex logic (not just simple assignment), such as loading configuration values, static data structures, or database drivers.

20. NON-STATIC (INSTANCE) BLOCKS

Q231. What is a non-static (instance) initializer block?

A: A block of code defined inside a class (without the static keyword) that runs every time an object is created, right before the constructor executes.

Q232. When does an instance block execute relative to the constructor?

A: The instance block executes immediately after the superclass constructor call completes but before the rest of the current class's constructor body executes.

Q233. What is the use case for instance initializer blocks?

A: To initialize instance variables with common logic shared across all constructors, especially useful when a class has multiple constructors requiring the same setup code.

Q234. Can a class have multiple instance blocks?

A: Yes, and they execute in the order they are written in the source code, each time an object is instantiated.

Q235. Do instance blocks run if you never create an object of the class?

A: No, instance blocks only run at the time of object creation (once per object created); if no object is created, they never execute (unlike static blocks, which run at class loading).

21. STATIC METHODS

Q236. What is a static method?

A: A method declared with the static keyword that belongs to the class rather than any instance, and can be called directly using the class name without creating an object.

Q237. Can a static method access instance variables directly?

A: No, a static method cannot directly access instance (non-static) variables or call instance methods because it doesn't operate in the context of any specific object.

Q238. Can a static method be overridden?

A: No, static methods cannot be truly overridden because they are resolved at compile time (static binding) based on reference type; if a subclass defines a static method with the same signature, it is called method hiding, not overriding.

Q239. Can a static method call another static method directly?

A: Yes, a static method can directly call other static methods of the same class without needing an object reference.

Q240. Can a static method be overloaded?

A: Yes, static methods can be overloaded like any other method, by varying the number, type, or order of parameters.

Q241. Why can't static methods use 'this' or 'super'?

A: Because 'this' and 'super' refer to a specific object instance, and static methods do not operate on any particular instance—they belong to the class as a whole.

Q242. Can a static method access a static variable?

A: Yes, static methods can freely access and modify static variables since both belong to the class rather than an instance.

22. WHY MAIN MUST BE STATIC

Q243. Why is the main() method declared static in Java?

A: Because the JVM needs to invoke main() without creating an object of the class first; if main() were non-static, the JVM would need an object to call it, but no object exists yet at program start, creating a chicken-and-egg problem.

Q244. What is the exact signature of the main method?

A: public static void main(String[] args)

Q245. Why is main() declared public?

A: So that the JVM (from outside the class/package) can access and invoke it to start program execution.

Q246. Why does main() have a String array (args) parameter?

A: To allow command-line arguments to be passed to the program at the time of execution.

Q247. Why is the return type of main() void?

A: Because main() doesn't need to return any value to the JVM; program termination and exit status are handled through other mechanisms (like System.exit()) if needed.

Q248. Can we change the signature of main() (e.g., remove static)?

A: If you remove static, add extra modifiers incorrectly, or change required elements, the program will compile (if syntactically valid) but the JVM will throw a runtime error (NoSuchMethodError) since it specifically looks for the standard main signature as an entry point.

Q249. Can the main method be overloaded?

A: Yes, but the JVM only calls the standard public static void main(String[] args); other overloaded versions must be invoked explicitly from within that main method or elsewhere.

23. ACCESSING NON-STATIC VARIABLES WITHIN STATIC CONTEXT

Q250. Why can't you directly access non-static (instance) variables inside a static method?

A: Because instance variables belong to individual objects, and a static method/context has no implicit association with any particular object—there's no guarantee an object even exists when the static method runs.

Q251. How can you access a non-static variable from within a static method?

A: By creating an object of the class inside the static method and accessing the instance variable through that object reference (e.g., ClassName obj = new ClassName(); obj.instanceVar).

Q252. What error occurs if you try to access an instance variable directly in a static context?

A: A compile-time error stating "non-static variable cannot be referenced from a static context."

Q253. Can a static method access a non-static method?

A: Only indirectly, by first creating an object and then calling the instance method on that object reference; it cannot call it directly by name.

Q254. Can a non-static (instance) method access static variables and methods?

A: Yes, instance methods can freely access static variables and static methods directly, since an instance always has access to class-level members.

24. INHERITANCE

Q255. What is inheritance in Java?

A: Inheritance is an OOP mechanism where a class (subclass/child) acquires the properties (fields) and behaviors (methods) of another class (superclass/parent), promoting code reusability.

Q256. What keyword is used to implement inheritance in Java?

A: The 'extends' keyword is used for class inheritance (and interface-to-interface inheritance); 'implements' is used when a class inherits from an interface.

Q257. What are the advantages of inheritance?

A: Code reusability, method overriding (enabling runtime polymorphism), logical hierarchical classification, and reduced redundancy.

Q258. What is a superclass and a subclass?

A: A superclass (parent/base class) is the class being inherited from; a subclass (child/derived class) is the class that inherits fields and methods from the superclass.

Q259. Does a subclass have access to private members of its superclass?

A: No, private members are not accessible directly by subclasses; they can only be accessed indirectly through public/protected getter/setter methods provided by the superclass.

Q260. Can a subclass access protected and public members of the superclass?

A: Yes, protected and public members are inherited and accessible in the subclass, whether in the same package or a different one (for protected, subclass access works even across packages).

Q261. Is a constructor inherited in Java?

A: No, constructors are not inherited, but a subclass constructor implicitly or explicitly invokes a superclass constructor via super().

Q262. What is the root class of all classes in Java?

A: The Object class (java.lang.Object) is the implicit superclass of every class in Java, either directly or indirectly.

25. TYPES OF INHERITANCE IN JAVA

Q263. What types of inheritance does Java support?

A: Single inheritance, multilevel inheritance, and hierarchical inheritance are directly supported through classes; multiple and hybrid inheritance are supported only through interfaces, not classes.

Q264. What is single inheritance?

A: When one subclass inherits from exactly one superclass (a simple parent-child relationship).

Q265. What is multilevel inheritance?

A: When a class is derived from a class which is itself derived from another class, forming a chain (e.g., Class C extends Class B, which extends Class A).

Q266. What is hierarchical inheritance?

A: When multiple subclasses inherit from a single superclass (one parent, many children).

Q267. What is multiple inheritance?

A: When a class inherits from more than one superclass simultaneously; Java does not support this for classes but supports it for interfaces (a class can implement multiple interfaces).

Q268. What is hybrid inheritance?

A: A combination of two or more types of inheritance (e.g., combining hierarchical and multiple); Java achieves this only via interfaces since multiple class inheritance is disallowed.

Q269. Does Java support multiple inheritance with classes?

A: No, Java does not allow a class to extend more than one class directly, to avoid ambiguity issues like the Diamond Problem.

26. WHY CYCLIC AND MULTIPLE INHERITANCE NOT ALLOWED

Q270. Why doesn't Java support multiple inheritance with classes?

A: To avoid the "Diamond Problem," where ambiguity arises if two parent classes have methods with the same signature, and the compiler cannot determine which version the child class should inherit.

Q271. What is the Diamond Problem?

A: It occurs when a class inherits from two classes that both have a method with the same signature; the subclass then has an ambiguous choice of which parent's method implementation to use, leading to conflicts.

Q272. How does Java resolve the Diamond Problem for interfaces (default methods)?

A: If a class implements two interfaces with conflicting default methods, the class is forced to explicitly override that method and specify which interface's version to use (via InterfaceName.super.methodName()), removing ambiguity.

Q273. Why is cyclic inheritance not allowed in Java?

A: Because it would create an infinite loop of inheritance (e.g., Class A extends Class B, and Class B extends Class A), which is logically impossible to resolve and is caught as a compile-time error by the compiler.

Q274. What error does the compiler show for cyclic inheritance?

A: It shows a compile-time error like "cyclic inheritance involving [ClassName]."

Q275. Can interfaces have cyclic inheritance?

A: No, similarly interfaces cannot cyclically extend one another; this also results in a compile-time error.

27. ORDER OF EXECUTION OF CHILD AND PARENT CONSTRUCTORS

Q276. What is the order of constructor execution in inheritance?

A: The parent (superclass) constructor always executes first, followed by the child (subclass) constructor, ensuring the parent's state is initialized before the child adds its own initialization.

Q277. Why does the parent constructor execute before the child constructor?

A: Because the child class may depend on fields/behavior initialized in the parent class, so the parent's state must be fully set up first for the child to build upon safely.

Q278. What happens implicitly if you don't call super() explicitly in a child constructor?

A: The compiler automatically inserts a call to super() (the no-argument parent constructor) as the first statement, provided the parent class has an accessible no-arg constructor.

Q279. What happens if the parent class has no no-argument constructor and the child doesn't explicitly call super(args)?

A: A compile-time error occurs because the compiler cannot implicitly insert a matching super() call; the child constructor must explicitly invoke the correct parameterized super(args).

Q280. Can a child class constructor execute without invoking any parent constructor?

A: No, every constructor call in Java (except for the Object class itself) implicitly or explicitly invokes a superclass constructor as its first action.

28. SUPER AND SUPER()

Q281. What is the 'super' keyword in Java?

A: 'super' is a reference variable used to refer to the immediate parent class's members (fields and methods) or to invoke the parent class's constructor.

Q282. What are the uses of 'super'?

A: To access parent class instance variables shadowed by the child, to call parent class methods (especially overridden ones), and to invoke the parent class constructor via super().

Q283. What is super() used for?

A: super() is used specifically to call the parent class's constructor, and if used, must be the first statement in the child's constructor.

Q284. Can super() be used to call a specific parameterized constructor of the parent?

A: Yes, super(arguments) can be used to explicitly invoke a matching parameterized constructor in the parent class.

Q285. Can 'super' be used in a static context?

A: No, 'super' cannot be used within static methods or blocks, since it refers to the parent instance, which requires an object context.

Q286. How do you call an overridden parent method from within the child class?

A: Using super.methodName(), which explicitly invokes the parent class's version of the method rather than the overridden child version.

Q287. What is the difference between 'this' and 'super'?

A: 'this' refers to the current class's instance, used to access current class members or call current class constructors, while 'super' refers to the immediate parent class's instance, used to access parent members or call parent constructors.

29. THIS AND THIS()

Q288. What is this() constructor call used for?

A: this() is used within a constructor to call another overloaded constructor of the same class, enabling constructor chaining and avoiding duplicate initialization code.

Q289. What is the rule regarding the placement of this() in a constructor?

A: this() must be the first statement in the constructor; it cannot appear anywhere else, and only one constructor call (this() or super()) is allowed per constructor.

Q290. Can this() and super() be used together in the same constructor?

A: No, since both must be the first statement, only one of them can be used in any single constructor—not both simultaneously.

Q291. What happens if this() creates a circular chain of constructor calls?

A: It results in a compile-time error, as the compiler detects the circular/recursive constructor invocation.

Q292. Give an example use case for this().

A: When a class has multiple constructors and one delegates common initialization logic to another, e.g., a no-arg constructor calling a parameterized one with default values: public Employee() { this("Unknown", 0); }

30. INHERITED METHODS

Q293. What are inherited methods?

A: Methods defined in a superclass that automatically become available to instances of the subclass without needing to be redefined, as long as they are not private.

Q294. Can a subclass use an inherited method directly?

A: Yes, a subclass object can call any public or protected (and default, if in the same package) method from the parent class as if it were defined in the subclass itself.

Q295. Are private methods of a parent class inherited?

A: No, private methods are not accessible/inherited by subclasses at all; they remain strictly scoped to the class in which they are declared.

Q296. Can inherited methods be used alongside newly defined methods in the subclass?

A: Yes, a subclass can freely mix inherited methods with its own newly declared methods, effectively extending the parent's functionality.

31. OVERRIDDEN METHODS

Q297. What is method overriding in Java?

A: Method overriding occurs when a subclass provides its own specific implementation of a method that is already defined in its superclass, with the same method signature (name, parameters, return type).

Q298. What are the rules for method overriding?

A: The method must have the same name and parameter list as in the parent; the return type must be the same or a covariant type; the access modifier cannot be more restrictive than the parent's; and it cannot override static, final, or private methods.

Q299. What is the @Override annotation used for?

A: It is an optional annotation that instructs the compiler to verify that the method is actually overriding a superclass method, catching errors at compile time if there's a signature mismatch.

Q300. Can a subclass override a final method?

A: No, methods declared as final in the parent class cannot be overridden by any subclass; attempting to do so causes a compile-time error.

Q301. Can a subclass override a static method?

A: No, static methods cannot be overridden (only hidden); if a subclass defines a static method with the same signature, it is a separate concept called method hiding, resolved at compile time.

Q302. What is covariant return type in method overriding?

A: It allows an overriding method in a subclass to return a type that is a subclass of the return type declared in the parent's method, instead of requiring an identical return type.

Q303. Can the access modifier be changed while overriding a method?

A: Yes, but it can only be widened (made less restrictive), not narrowed—e.g., a protected method in the parent can be overridden as public, but not as private.

Q304. What is dynamic method dispatch?

A: It is the mechanism by which a call to an overridden method is resolved at runtime based on the actual object type (not the reference type), enabling runtime polymorphism.

Q305. Can constructors be overridden?

A: No, constructors cannot be overridden since they are not inherited and are tied to their own class name.

32. SPECIALIZED METHODS

Q306. What is meant by "specialized methods" in the context of inheritance?

A: These are methods that exist only in the subclass and are not present in the parent class, providing behavior unique/specific to the child class beyond what it inherits.

Q307. Can a superclass reference call a specialized (subclass-only) method?

A: No, a superclass reference variable can only call methods that are defined in the superclass (or overridden ones); to call subclass-specific methods, the reference must be explicitly downcast to the subclass type.

Q308. How do you call a subclass-specific method using a parent class reference?

A: By downcasting the reference to the subclass type first, e.g., ((ChildClass) parentRef).specializedMethod();, ideally after checking with instanceof.

Q309. Why is it useful to have specialized methods in subclasses?

A: They allow each subclass to extend the base functionality with behavior relevant only to that specific type, supporting more granular and specific object behavior within a class hierarchy.

33. PARENT REFERENCE HOLDING CHILD OBJECT

Q310. What does it mean when a parent reference holds a child object?

A: It means a variable of the superclass type is assigned an instance of a subclass, e.g., Animal a = new Dog();, which is a valid and common practice called upcasting.

Q311. What members can be accessed when a parent reference holds a child object?

A: Only members (fields and methods) defined in the parent class type are accessible directly through that reference; however, for overridden methods, the actual (child) implementation executes due to dynamic method dispatch.

Q312. Why is this concept (parent ref holding child object) important?

A: It is fundamental to achieving runtime polymorphism, allowing a single reference type to behave differently based on the actual object it points to, which is key for flexible and extensible code design.

Q313. Can you access subclass-only fields/methods via a parent reference without casting?

A: No, attempting to access subclass-specific members directly through a parent reference (without casting) results in a compile-time error, since the compiler only knows about the reference's declared type.

Q314. What is the difference in behavior between fields and overridden methods when accessed via a parent reference to a child object?

A: Fields are resolved based on the reference type (static binding, no polymorphism for fields), while overridden methods are resolved based on the actual object type at runtime (dynamic binding/polymorphism).

34. RUNTIME POLYMORPHISM

Q315. What is runtime polymorphism in Java?

A: Also known as dynamic polymorphism, it is the ability of an object to take multiple forms, resolved at runtime through method overriding, where the JVM decides which overridden method to invoke based on the actual object type.

Q316. How is runtime polymorphism achieved in Java?

A: Through method overriding combined with upcasting (a parent class reference pointing to a child class object), where the specific overridden method called depends on the actual runtime object.

Q317. What is the difference between compile-time and runtime polymorphism?

A: Compile-time polymorphism (method overloading) is resolved by the compiler based on method signatures at compile time; runtime polymorphism (method overriding) is resolved by the JVM at runtime based on the actual object type.

Q318. What is dynamic method dispatch in relation to runtime polymorphism?

A: It is the underlying mechanism that enables runtime polymorphism—the JVM looks up and calls the appropriate overridden method implementation based on the object's actual class at the time of the call, not the reference type.

Q319. Can runtime polymorphism be achieved with static methods?

A: No, since static methods are resolved at compile time (static binding) based on reference type, not dynamically at runtime, so runtime polymorphism does not apply to them.

Q320. What is the real-world benefit of runtime polymorphism?

A: It allows writing flexible, extensible code where new subclasses can be introduced without modifying existing code that operates on the parent type, adhering to the Open/Closed Principle.

35. ACCESS MODIFIERS IN JAVA (PUBLIC, PROTECTED, DEFAULT, PRIVATE)

Q321. What are access modifiers in Java?

A: Access modifiers are keywords that set the accessibility/visibility of classes, methods, constructors, and fields to other classes.

Q322. What are the four access modifiers in Java?

A: public, protected, default (no modifier/package-private), and private.

Q323. What is the scope of the public access modifier?

A: Public members are accessible from anywhere—within the same class, same package, subclasses, and any other package.

Q324. What is the scope of the private access modifier?

A: Private members are accessible only within the same class in which they are declared; not accessible even to subclasses or other classes in the same package.

Q325. What is the scope of the protected access modifier?

A: Protected members are accessible within the same package and also in subclasses located in different packages (via inheritance).

Q326. What is the scope of the default (package-private) access modifier?

A: When no access modifier is specified, the member is accessible only within classes in the same package, and not accessible from outside the package (even by subclasses in other packages).

Q327. Can a top-level class be declared private or protected?

A: No, a top-level class can only be declared as public or default (package-private); private and protected are not valid for top-level classes (only for nested/inner classes and class members).

Q328. Which access modifier provides the most restrictive access? Which is the least restrictive?

A: private is the most restrictive (class-only access); public is the least restrictive (accessible everywhere). Order from most to least restrictive: private < default < protected < public.

Q329. Can a subclass in a different package access a protected member of its superclass?

A: Yes, but only through inheritance (i.e., within the subclass's own code referring to its inherited member or via a subclass-type reference), not through an arbitrary instance of the parent class held by unrelated code in that other package.

Q330. What access modifiers can be applied to constructors?

A: All four—public, protected, default, and private—can be applied to constructors, controlling how/where the class can be instantiated.

36. APPLICATION OF FINAL

Q331. What does the 'final' keyword mean in Java?

A: 'final' is a non-access modifier used to restrict modification—it can be applied to variables (making them constants), methods (preventing overriding), and classes (preventing inheritance).

Q332. What happens when a variable is declared final?

A: Its value can be assigned only once; any attempt to reassign it afterward results in a compile-time error. It effectively becomes a constant.

Q333. What is a blank final variable?

A: A final variable that is declared but not initialized at the point of declaration; it must be initialized exactly once, either in a constructor or an instance initializer block, before the object is fully constructed.

Q334. What happens when a method is declared final?

A: It cannot be overridden by any subclass; attempting to override a final method results in a compile-time error.

Q335. What happens when a class is declared final?

A: The class cannot be extended/inherited by any other class; attempting to do so results in a compile-time error. Example: the String class is final.

Q336. Can a final class have subclasses?

A: No, by definition, a final class cannot be extended, so it cannot have any subclasses.

Q337. Can final variables be static?

A: Yes, a variable can be both static and final simultaneously, commonly used to define class-level constants (e.g., public static final double PI = 3.14;).

Q338. Can constructors be declared final?

A: No, since constructors are not inherited in the first place, marking them final is not applicable and results in a compile-time error.

Q339. Is a final reference variable's object also immutable?

A: No, a final reference variable means the reference itself cannot be reassigned to point to a different object, but the internal state of the object it points to can still be modified (unless the object's class is itself designed to be immutable).

Q340. Why would you use final for method parameters?

A: To prevent the parameter's value from being reassigned within the method body, which can help avoid accidental modification and clarify intent, especially useful in lambda expressions requiring effectively final variables.

37. COMPOSITION AND AGGREGATION

Q341. What is composition in Java (OOP)?

A: Composition is a "has-a" relationship where one class contains an object of another class as a field, and the contained object's lifecycle is tightly bound to the containing object (it cannot exist independently).

Q342. What is aggregation in Java (OOP)?

A: Aggregation is also a "has-a" relationship, but a weaker one, where the contained object can exist independently of the containing object's lifecycle (e.g., a Department can exist even if a specific Employee object is removed).

Q343. What is the key difference between composition and aggregation?

A: In composition, the child object's existence depends entirely on the parent (strong ownership, lifecycle tightly coupled); in aggregation, the child object can exist independently of the parent (weaker ownership).

Q344. Give an example of composition.

A: A House class containing a Room object as a field—if the House is destroyed, the Room object it created and owns has no independent existence (e.g., House creates and manages its own Room instances internally).

Q345. Give an example of aggregation.

A: A Department class holding a list of Employee objects—if the Department is deleted, the Employee objects can still exist independently (perhaps assigned to another department).

Q346. Why is composition often preferred over inheritance ("favor composition over inheritance")?

A: Composition provides greater flexibility (behavior can change at runtime by swapping composed objects), avoids issues of tight coupling and fragile base class problems associated with deep inheritance hierarchies, and better models "has-a" relationships versus "is-a".

Q347. How is composition implemented in Java code?

A: By declaring an instance of another class as a field within a class, typically initialized within the containing class's own constructor.

38. INTERFACE IN JAVA

Q348. What is an interface in Java?

A: An interface is a reference type, similar to a class, that can contain abstract methods (implicitly public and abstract), default methods, static methods, and constants (implicitly public, static, and final); it defines a contract that implementing classes must fulfill.

Q349. How is an interface declared and implemented?

A: Declared using the 'interface' keyword; a class uses 'implements' to provide implementations for the interface's abstract methods.

Q350. Can an interface have a constructor?

A: No, interfaces cannot have constructors because they cannot be instantiated directly.

Q351. Can a class implement multiple interfaces?

A: Yes, unlike class inheritance, a Java class can implement multiple interfaces simultaneously, which is how Java achieves a form of multiple inheritance.

Q352. What are default methods in an interface?

A: Introduced in Java 8, default methods are interface methods with a concrete implementation (using the 'default' keyword), allowing interfaces to add new methods without breaking existing implementing classes.

Q353. What are static methods in an interface?

A: Also introduced in Java 8, these are methods with implementation defined directly in the interface and are called using the interface name (InterfaceName.method()), not inherited by implementing classes.

Q354. Can an interface extend another interface?

A: Yes, and unlike classes, an interface can extend multiple other interfaces simultaneously using the 'extends' keyword.

Q355. Are variables in an interface implicitly static and final?

A: Yes, all variables declared in an interface are implicitly public, static, and final (constants), and must be initialized at the point of declaration.

Q356. Can an interface have private methods?

A: Yes, since Java 9, interfaces can have private methods (and private static methods), used to share common code between default/static methods within the interface itself, not accessible outside.

Q357. What happens if a class implements two interfaces with the same default method signature?

A: The implementing class must override that method explicitly to resolve the conflict, optionally calling a specific interface's version using InterfaceName.super.methodName().

Q358. What is a functional interface?

A: An interface that contains exactly one abstract method (though it may have multiple default/static methods), often annotated with @FunctionalInterface and used as the target type for lambda expressions.

Q359. Can an interface be instantiated?

A: No, an interface cannot be directly instantiated using 'new'; however, an anonymous class implementing the interface can be created, or (since Java 8) an interface with default methods only could theoretically not need implementation but still can't be 'new'-ed directly.

Q360. Why use interfaces instead of abstract classes?

A: Interfaces allow a class to achieve multiple inheritance of type, define pure contracts/capabilities without imposing a class hierarchy, and are ideal when unrelated classes need to share a common behavior contract.

39. ABSTRACT CLASSES AND METHODS

Q361. What is an abstract class in Java?

A: An abstract class is a class declared with the 'abstract' keyword that cannot be instantiated directly and may contain both abstract methods (without implementation) and concrete methods (with implementation).

Q362. What is an abstract method?

A: A method declared without a body/implementation, using the 'abstract' keyword, that must be implemented (overridden) by any concrete (non-abstract) subclass.

Q363. Can an abstract class have a constructor?

A: Yes, abstract classes can have constructors, which are invoked when a subclass object is instantiated (via super()), even though the abstract class itself cannot be directly instantiated.

Q364. Can an abstract class have non-abstract (concrete) methods?

A: Yes, an abstract class can freely mix abstract and fully-implemented (concrete) methods.

Q365. What happens if a subclass doesn't implement all abstract methods of its abstract parent class?

A: The subclass must also be declared abstract itself; otherwise, a compile-time error occurs since a concrete class must implement all inherited abstract methods.

Q366. Can an abstract class have static methods?

A: Yes, abstract classes can have static methods, which can be called directly using the class name, independent of any subclass instance.

Q367. Can a class be both abstract and final?

A: No, this combination is contradictory and causes a compile-time error—abstract requires subclassing to be useful, while final prevents subclassing entirely.

Q368. What is the difference between an abstract class and an interface?

A: An abstract class can have constructors, instance fields with any access modifier, and a mix of abstract/concrete methods, and supports single inheritance; an interface cannot have constructors or instance state (only constants), traditionally only abstract methods (though now allows default/static/private methods), and supports multiple inheritance.

Q369. When would you choose an abstract class over an interface?

A: When you need to share common state (instance variables) and code among closely related classes, or need constructors, or want to provide a partial default implementation while enforcing certain methods to be defined by subclasses.

Q370. Can an abstract method be private?

A: No, an abstract method cannot be private because private methods aren't inherited/visible to subclasses, and the entire point of an abstract method is for subclasses to override and implement it.

Q371. Can an abstract method be static?

A: No, static methods are resolved at compile time (not overridden), so declaring an abstract method (which requires subclass implementation via overriding) as static is contradictory and not allowed.

Q372. Can an abstract class implement an interface without implementing its methods?

A: Yes, an abstract class is allowed to implement an interface without providing implementations for the interface's abstract methods, leaving that responsibility to its own concrete subclasses.

40. EXCEPTION HANDLING

Q373. What is an exception in Java?

A: An exception is an event that disrupts the normal flow of a program's execution, represented as an object that is thrown at runtime when an error condition occurs.

Q374. What is exception handling?

A: It is the mechanism to handle runtime errors gracefully using try, catch, finally, throw, and throws, preventing abrupt program termination and allowing controlled recovery or reporting.

Q375. What is the try-catch block?

A: A try block contains code that might throw an exception; a catch block, placed immediately after, catches and handles a specific type of exception if it occurs within the try block.

Q376. Can you have multiple catch blocks for a single try block?

A: Yes, multiple catch blocks can be used to handle different types of exceptions separately for the same try block.

Q377. What is a multi-catch block (Java 7+)?

A: A single catch block that can handle multiple exception types using the pipe (|) symbol, e.g., catch (IOException | SQLException e) { }, reducing code duplication.

Q378. What happens if no matching catch block is found for a thrown exception?

A: The exception propagates up the call stack to the calling method; if it remains unhandled all the way up, the JVM terminates the program and prints the stack trace.

Q379. Can a try block exist without a catch block?

A: Yes, provided it is followed by a finally block (try-finally), which is valid even without any catch block.

Q380. What happens to code inside the try block after an exception is thrown mid-way?

A: Execution immediately stops at the point of the exception, skips the remaining code in the try block, and control jumps to the matching catch block (if any).

41. DUCKING VS RETHROWING EXCEPTION

Q381. What does "ducking" an exception mean?

A: Ducking refers to not handling an exception within the current method and instead declaring it with 'throws' in the method signature, letting the calling method handle it.

Q382. What is rethrowing an exception?

A: Rethrowing is when a catch block catches an exception, performs some action (like logging), and then explicitly throws the same (or a new/wrapped) exception again using the 'throw' keyword, passing responsibility further up the call stack.

Q383. What is the difference between ducking and rethrowing?

A: Ducking avoids handling the exception entirely in the current method (just declares it with throws); rethrowing involves catching the exception first (allowing some processing/logging) before deliberately throwing it again.

Q384. Why would a developer choose to rethrow an exception instead of fully handling it?

A: To perform partial handling (like logging or cleanup) at the current level while still allowing higher-level code (which may have more context) to make the final decision on how to fully resolve or report the error.

Q385. What is exception wrapping/chaining?

A: The practice of catching an exception and throwing a new exception that wraps the original as its "cause," preserving the original stack trace information while providing more context-specific error information.

42. WHY JAVA IS A ROBUST LANGUAGE

Q386. Why is Java considered a robust language?

A: Because of features like strong compile-time type checking, automatic memory management (garbage collection), robust exception handling mechanisms, absence of explicit pointers (reducing memory corruption risks), and array bounds checking at runtime.

Q387. How does exception handling contribute to Java's robustness?

A: It allows programs to handle runtime errors gracefully without crashing entirely, enabling recovery, cleanup, and meaningful error reporting instead of abrupt termination.

Q388. How does garbage collection contribute to robustness?

A: It automatically manages memory deallocation, reducing memory leaks and dangling pointer errors that commonly occur in languages requiring manual memory management like C/C++.

Q389. How does the absence of explicit pointers contribute to robustness?

A: It prevents direct memory manipulation errors (like pointer arithmetic mistakes) that can lead to memory corruption, security vulnerabilities, and crashes common in pointer-based languages.

Q390. How does strong type checking contribute to robustness?

A: Java enforces type checking both at compile time and runtime, catching many errors early before the program runs and preventing invalid type operations from causing unpredictable behavior.

43. ERROR VS EXCEPTION

Q391. What is the difference between an Error and an Exception in Java?

A: Errors represent serious problems typically related to the environment/JVM (like OutOfMemoryError or StackOverflowError) that applications usually cannot recover from, while Exceptions represent conditions that a program might want to catch and handle.

Q392. Are Errors and Exceptions both subclasses of the same class?

A: Yes, both Error and Exception are subclasses of the Throwable class, which is the root class for all throwable conditions in Java.

Q393. Should you catch Errors in your code?

A: Generally, no—Errors indicate serious, often unrecoverable problems (like JVM issues), and are not meant to be caught and handled by application-level code, unlike Exceptions.

Q394. Give examples of common Errors in Java.

A: OutOfMemoryError, StackOverflowError, NoClassDefFoundError, and AssertionError.

Q395. Give examples of common Exceptions in Java.

A: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException, IOException, and NumberFormatException.

44. ORDER OF CATCH BLOCKS

Q396. What is the rule regarding the order of catch blocks?

A: More specific exception types must be caught before more general ones; a subclass exception's catch block must come before its superclass exception's catch block.

Q397. What happens if you place a superclass exception's catch block before a subclass exception's catch block?

A: It causes a compile-time error ("exception X has already been caught") because the broader catch block would always match first, making the subsequent, more specific catch block unreachable.

Q398. Where should the catch block for the generic Exception class be placed?

A: Since Exception is the superclass of most exception types, its catch block should be placed last, after all more specific exception catch blocks.

Q399. Can you catch Throwable in a catch block?

A: Yes, though it's generally discouraged, since catching Throwable also catches Errors, which typically shouldn't be handled at the application level.

45. EXCEPTION CLASS

Q400. What is the Exception class in Java?

A: Exception is a class in java.lang package that represents conditions a reasonable application might want to catch; it is the superclass for all checked exceptions and the parent of RuntimeException (which covers unchecked exceptions).

Q401. What is the class hierarchy starting from Throwable?

A: Throwable is the root class, with two direct subclasses: Error and Exception. Exception further has RuntimeException as a subclass (covering unchecked exceptions), while other direct Exception subclasses are checked exceptions.

Q402. What methods does the Exception class provide?

A: getMessage(), getCause(), printStackTrace(), toString(), and getStackTrace(), inherited largely from Throwable.

Q403. Can you create your own exception class? How?

A: Yes, by extending Exception (for a checked exception) or RuntimeException (for an unchecked exception), typically providing constructors that call the super class constructors with a message and/or cause.

46. FINALLY BLOCK

Q404. What is the finally block in Java?

A: The finally block is an optional block that follows try/catch and always executes regardless of whether an exception was thrown or caught, typically used for cleanup operations like closing resources.

Q405. Does the finally block always execute?

A: Almost always—yes, even if an exception is thrown, caught, or not caught, or if the try/catch block has a return statement. The only exceptions are if System.exit() is called, the JVM crashes, or the thread is forcibly killed.

Q406. What happens if both the try block and the finally block have a return statement?

A: The return statement in the finally block overrides/suppresses the one in the try block; the method ultimately returns the value specified in finally.

Q407. Can a finally block exist without a catch block?

A: Yes, a try-finally combination (without any catch block) is valid; the finally block still executes after the try block, regardless of exceptions.

Q408. What is the primary use case of the finally block?

A: Releasing resources such as closing file streams, database connections, or network sockets, ensuring cleanup happens whether or not an exception occurred.

Q409. If an exception is thrown inside the finally block itself, what happens?

A: It can suppress or replace any exception that was already being propagated from the try/catch blocks, potentially masking the original exception—this is generally considered bad practice.

Q410. What is try-with-resources, and how does it relate to finally?

A: Introduced in Java 7, try-with-resources automatically closes resources (that implement AutoCloseable) at the end of the try block, eliminating the need for explicit finally blocks to close resources manually.

47. CUSTOM EXCEPTIONS

Q411. What is a custom (user-defined) exception in Java?

A: A custom exception is a class created by the programmer, typically extending Exception or RuntimeException, to represent application-specific error conditions not covered by Java's built-in exceptions.

Q412. How do you create a custom checked exception?

A: By extending the Exception class and providing constructors, e.g., class InsufficientFundsException extends Exception { public InsufficientFundsException(String message) { super(message); } }

Q413. How do you create a custom unchecked exception?

A: By extending RuntimeException instead of Exception, following the same constructor pattern, resulting in an exception that doesn't need to be declared or explicitly caught.

Q414. Why would you create a custom exception instead of using a built-in one?

A: To provide more meaningful, domain-specific error information, improve code readability, and allow for more precise exception handling tailored to specific business logic scenarios.

Q415. What is a common naming convention for custom exception classes?

A: Custom exception class names typically end with the word "Exception," e.g., InvalidAgeException, OutOfStockException.

48. getMessage() AND printStackTrace()

Q416. What does the getMessage() method do?

A: It returns the detail message string of a Throwable object, which was provided when the exception was constructed (or null if none was provided).

Q417. What does printStackTrace() do?

A: It prints the exception's type, message, and the full stack trace (the sequence of method calls that led to the exception) to the standard error stream, useful for debugging.

Q418. What is the difference between getMessage() and toString() on an exception?

A: getMessage() returns just the detail message string; toString() returns a more complete representation including the exception's fully qualified class name concatenated with the message.

Q419. What does getStackTrace() return?

A: It returns an array of StackTraceElement objects representing the call stack at the point the exception was created, which can be used to programmatically inspect the trace rather than just print it.

Q420. What does getCause() do?

A: It returns the underlying cause of the exception (another Throwable), if the exception was created with a cause (used for exception chaining); returns null if no cause was set.

49. CHECKED VS UNCHECKED EXCEPTIONS

Q421. What are checked exceptions?

A: Checked exceptions are exceptions that are checked at compile time; the compiler forces the programmer to either handle them (try-catch) or declare them (throws) in the method signature. They extend Exception but not RuntimeException.

Q422. What are unchecked exceptions?

A: Unchecked exceptions are exceptions not checked at compile time; the compiler does not force handling or declaration. They extend RuntimeException (or Error), and typically indicate programming bugs.

Q423. Give examples of checked exceptions.

A: IOException, SQLException, ClassNotFoundException, FileNotFoundException.

Q424. Give examples of unchecked exceptions.

A: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException, NumberFormatException, IllegalArgumentException.

Q425. Why does Java distinguish between checked and unchecked exceptions?

A: Checked exceptions represent recoverable conditions that a well-written application should anticipate and handle (like file not found), while unchecked exceptions typically represent programming errors/bugs that shouldn't normally occur if the code is correct.

Q426. Is a class required to declare unchecked exceptions using 'throws'?

A: No, since unchecked exceptions are not verified by the compiler, there's no requirement to declare them in the method signature, though it can still be done optionally for documentation purposes.

Q427. Where do checked and unchecked exceptions fit in the exception hierarchy?

A: Both are subclasses of Exception; unchecked exceptions specifically extend RuntimeException (a subclass of Exception), while checked exceptions are any Exception subclass that does NOT extend RuntimeException.

50. THROWABLE

Q428. What is the Throwable class?

A: Throwable is the superclass of all errors and exceptions in Java; only objects of this class (or its subclasses) can be thrown using the 'throw' keyword or caught using 'catch'.

Q429. What are the two direct subclasses of Throwable?

A: Error and Exception.

Q430. What methods are commonly available on the Throwable class?

A: getMessage(), getLocalizedMessage(), getCause(), printStackTrace(), getStackTrace(), toString(), and initCause().

Q431. Can you catch a Throwable directly?

A: Yes, technically you can write catch (Throwable t) { }, which would catch both Exceptions and Errors, though this is generally discouraged as bad practice.

51. THROW VS THROWS

Q432. What is the 'throw' keyword used for?

A: 'throw' is used to explicitly throw an exception instance from within a method or block of code, e.g., throw new IllegalArgumentException("Invalid input");

Q433. What is the 'throws' keyword used for?

A: 'throws' is used in a method signature to declare that the method might throw one or more checked exceptions, informing callers that they need to handle or further propagate them.

Q434. What is the key difference between 'throw' and 'throws'?

A: 'throw' is used to actually throw a single exception instance at a specific point in the code; 'throws' is used in a method declaration to list potential exception types the method may propagate, without actually throwing anything itself.

Q435. Can 'throw' be used to throw multiple exceptions at once?

A: No, 'throw' can only throw one exception object at a time per statement, since execution halts immediately after the throw statement executes.

Q436. Can 'throws' declare multiple exceptions?

A: Yes, a method can declare multiple exception types separated by commas in its 'throws' clause, e.g., public void readFile() throws IOException, SQLException { }

Q437. What is required after a 'throw' statement in terms of code reachability?

A: Since 'throw' immediately transfers control (like a return statement), any code placed immediately after an unconditional throw statement within the same block is unreachable and causes a compile-time error.

Q438. Is it mandatory to declare unchecked exceptions using 'throws'?

A: No, it's optional; unchecked exceptions are not checked by the compiler, so declaring them with 'throws' is not required (though it can be done for clarity/documentation).

52. MULTITHREADING

Q439. What is multithreading in Java?

A: Multithreading is the capability of a program to execute multiple threads (lightweight sub-processes) concurrently, allowing simultaneous execution of multiple parts of a program to improve performance and resource utilization.

Q440. What is a thread?

A: A thread is the smallest unit of execution within a process, having its own call stack but sharing the same memory (heap) space with other threads of the same process.

Q441. What is the difference between a process and a thread?

A: A process is an independent program with its own memory space, while a thread is a lightweight sub-unit of a process that shares the process's memory space with other threads but has its own stack and program counter.

Q442. What are the two main ways to create a thread in Java?

A: By extending the Thread class and overriding its run() method, or by implementing the Runnable interface and passing an instance to a Thread object.

Q443. What is the run() method?

A: It is the method that contains the code to be executed by the thread; it must be overridden (from Thread or implemented from Runnable) to define the thread's task.

Q444. What is concurrency versus parallelism?

A: Concurrency is the ability to manage multiple tasks by interleaving their execution (may or may not run simultaneously, e.g., on a single core via context switching), while parallelism means tasks are actually executed simultaneously on multiple processor cores.

53. THREAD SCHEDULER

Q445. What is a thread scheduler in Java?

A: The thread scheduler is part of the JVM (relying on the underlying OS) that determines the order in which runnable threads are executed and allocated CPU time, based on thread priority and scheduling algorithms.

Q446. Does Java guarantee a specific thread scheduling algorithm?

A: No, thread scheduling behavior is JVM/OS-dependent and not guaranteed to follow a specific algorithm (like round-robin or priority-based); actual behavior can vary across platforms.

Q447. What role does thread priority play in scheduling?

A: Thread priority is a hint (ranging from 1 to 10, default 5) given to the scheduler about the relative importance of threads, but the scheduler is not obligated to strictly honor it—it's platform-dependent.

Q448. How do you set thread priority in Java?

A: Using the setPriority(int) method, with values ranging from Thread.MIN_PRIORITY (1) to Thread.MAX_PRIORITY (10), and Thread.NORM_PRIORITY (5) as default.

Q449. What is context switching?

A: Context switching is the process where the CPU switches from executing one thread to another, saving the state of the current thread and loading the state of the next thread to be executed.

54. MULTITHREAD AND MULTIPLE STACKS

Q450. Why does each thread have its own stack?

A: Each thread needs its own call stack to independently track its own method calls, local variables, and execution state, since threads execute code concurrently and need isolated execution contexts.

Q451. Do threads share heap memory?

A: Yes, all threads within the same process share the same heap memory, which is why objects can be accessed and modified by multiple threads, potentially leading to concurrency issues if not synchronized.

Q452. What is stored in a thread's stack versus the shared heap?

A: A thread's stack stores local variables, method call frames, and partial results specific to that thread; the shared heap stores objects and instance/static data accessible by all threads.

Q453. Why can concurrent access to heap objects cause problems, while stack data is generally safe?

A: Because heap objects are shared among all threads, simultaneous read/write access without proper synchronization can lead to race conditions and inconsistent data, whereas each thread's own stack data is private and inherently thread-safe.

55. EXTENDS THREAD VS IMPLEMENTS RUNNABLE

Q454. What are the two ways to define a thread's task in Java?

A: By extending the Thread class and overriding run(), or by implementing the Runnable interface and passing an instance of it to a Thread object's constructor.

Q455. What is the key difference between extending Thread and implementing Runnable?

A: Extending Thread uses up the single inheritance slot in Java (the class cannot extend any other class), while implementing Runnable leaves the class free to extend another class since interfaces support multiple implementation.

Q456. Which approach is generally recommended: extends Thread or implements Runnable?

A: Implementing Runnable is generally preferred because it promotes better design (separation of task logic from thread management), avoids wasting the single inheritance opportunity, and allows the same Runnable instance to be shared across multiple threads.

Q457. How do you start a thread created using the Runnable interface?

A: By passing the Runnable instance to a Thread object's constructor and then calling start() on the Thread object, e.g., new Thread(runnableInstance).start();

Q458. Can a class both extend Thread and implement Runnable?

A: Technically yes, but it's redundant and unconventional since Thread itself already implements Runnable; there's no practical benefit to doing both.

Q459. What happens if you call run() directly instead of start()?

A: Calling run() directly executes the method as a normal synchronous method call on the current thread, without creating a new thread; it does not achieve any actual concurrency.

56. START()

Q460. What does the start() method do?

A: start() creates a new thread of execution and calls the run() method on that new thread asynchronously; it is the correct way to begin concurrent execution.

Q461. What is the difference between start() and run()?

A: start() creates a new thread and the JVM invokes run() on it, enabling true concurrent execution; calling run() directly just executes the method like any normal method call on the current thread, without spawning a new thread.

Q462. Can start() be called more than once on the same thread object?

A: No, calling start() a second time on the same thread that has already been started (or completed) throws an IllegalThreadStateException.

Q463. What happens internally when start() is invoked?

A: The JVM allocates necessary resources for a new thread, registers it with the thread scheduler, and once scheduled, the thread's run() method begins execution independently of the calling thread.

57. DAEMON THREADS

Q464. What is a daemon thread in Java?

A: A daemon thread is a low-priority background thread that runs to support user threads (e.g., garbage collector); the JVM automatically terminates all daemon threads once all user (non-daemon) threads finish execution.

Q465. How do you make a thread a daemon thread?

A: Using the setDaemon(true) method before calling start() on the thread; calling it after the thread has started throws an IllegalThreadStateException.

Q466. What is a common example of a daemon thread in the JVM?

A: The Garbage Collector thread is a classic example of a JVM-managed daemon thread.

Q467. Does the JVM wait for daemon threads to finish before exiting?

A: No, the JVM exits/terminates once all user (non-daemon) threads have finished, regardless of whether daemon threads are still running; daemon threads are abruptly stopped.

Q468. How do you check if a thread is a daemon thread?

A: Using the isDaemon() method, which returns a boolean.

58. RACE CONDITION

Q469. What is a race condition in multithreading?

A: A race condition occurs when two or more threads access shared data concurrently, and the final outcome depends on the unpredictable timing/order of their execution, potentially leading to incorrect or inconsistent results.

Q470. How can race conditions be prevented?

A: By using synchronization mechanisms (synchronized keyword, locks) to ensure that only one thread can access/modify the shared resource at a time, or by using thread-safe data structures/atomic variables.

Q471. Give a simple example scenario of a race condition.

A: Two threads simultaneously incrementing a shared counter variable (count++) without synchronization; since increment is not atomic (read-modify-write), both threads might read the same initial value, leading to a lost update and an incorrect final count.

Q472. What is a critical section in the context of race conditions?

A: A critical section is the part of code that accesses shared resources and must not be executed by more than one thread at the same time to prevent race conditions.

59. DEADLOCKS

Q473. What is a deadlock in Java multithreading?

A: A deadlock is a situation where two or more threads are blocked forever, each waiting for a resource/lock held by the other, resulting in none of them being able to proceed.

Q474. What are the necessary conditions for a deadlock to occur?

A: Mutual exclusion (resources can't be shared), hold and wait (a thread holds a resource while waiting for another), no preemption (resources can't be forcibly taken away), and circular wait (a cycle of threads waiting on each other).

Q475. How can deadlocks be avoided in Java?

A: By always acquiring locks in a consistent, predefined order across all threads, using timeout mechanisms (like tryLock()), minimizing the scope/duration of locks, and avoiding nested locks where possible.

Q476. Give a simple example of a deadlock scenario.

A: Thread A locks Object 1 and waits to lock Object 2, while simultaneously Thread B locks Object 2 and waits to lock Object 1—neither can proceed since each holds what the other needs.

Q477. Does Java provide built-in deadlock detection?

A: No automatic runtime recovery, but tools like jstack or VisualVM can be used to analyze thread dumps and detect deadlocks for debugging purposes; the JVM does not automatically resolve deadlocks.

60. SYNCHRONIZATION

Q478. What is synchronization in Java?

A: Synchronization is a mechanism that controls access to shared resources by multiple threads, ensuring that only one thread can execute a synchronized method/block on a given object at a time, preventing race conditions.

Q479. What keyword is used to achieve synchronization?

A: The 'synchronized' keyword, which can be applied to methods or specific blocks of code.

Q480. What is a monitor lock (intrinsic lock) in Java?

A: Every object in Java has an associated intrinsic lock (monitor); when a thread enters a synchronized method/block on an object, it acquires that object's monitor lock, and releases it

upon exiting, preventing other threads from entering synchronized sections on the same object simultaneously.

Q481. What are the downsides of excessive synchronization?

A: It can reduce performance due to threads waiting/blocking for locks, and can lead to issues like deadlocks if not carefully designed.

61. SYNCHRONIZED METHODS VS BLOCKS

Q482. What is a synchronized method?

A: A method declared with the 'synchronized' keyword, which locks the entire method using the object's (or class's, for static methods) intrinsic lock, allowing only one thread to execute that method on that object at a time.

Q483. What is a synchronized block?

A: A block of code within a method wrapped with synchronized(object) { }, allowing finer-grained control by locking only a specific section of code rather than the entire method, using a specified object's lock.

Q484. What is the advantage of a synchronized block over a synchronized method?

A: Synchronized blocks provide better performance by locking only the critical section of code that truly needs synchronization, rather than the entire method, reducing contention and increasing concurrency.

Q485. How do static synchronized methods differ from instance synchronized methods in terms of locking?

A: A static synchronized method locks on the Class object (shared across all instances), while an instance synchronized method locks on the specific object instance (this), meaning they use different locks and don't block each other.

Q486. Can you specify which object's lock to use in a synchronized block?

A: Yes, synchronized(someObject) { } explicitly specifies which object's monitor lock to acquire, providing flexibility beyond just using 'this' or the class object.

62. JOIN()

Q487. What does the join() method do?

A: join() causes the calling thread to pause and wait until the thread on which join() was called completes its execution (or a specified timeout elapses).

Q488. Give an example use case for join().

A: When a main thread needs to wait for a worker thread to finish processing data before proceeding to use the results, e.g., workerThread.join(); ensures the main thread waits.

Q489. Can join() accept a timeout parameter?

A: Yes, join(long millis) makes the calling thread wait for at most the specified time; if the thread hasn't finished by then, execution continues anyway.

Q490. What exception can join() throw?

A: InterruptedException, a checked exception, if another thread interrupts the waiting thread while it's blocked in join().

63. SLEEP()

Q491. What does Thread.sleep() do?

A: It pauses/suspends the execution of the currently executing thread for a specified duration (in milliseconds, optionally with nanoseconds), without releasing any locks it holds.

Q492. Is sleep() a static or instance method?

A: sleep() is a static method of the Thread class, meaning it always causes the currently executing thread to pause, regardless of which Thread object it's called on syntactically.

Q493. Does sleep() release the lock held by the thread?

A: No, unlike wait(), sleep() does not release any monitor locks the thread currently holds; the thread simply pauses while retaining its locks.

Q494. What exception does sleep() throw?

A: InterruptedException, a checked exception, thrown if another thread interrupts the sleeping thread.

64. YIELD()

Q495. What does Thread.yield() do?

A: yield() is a static method that provides a hint to the thread scheduler that the current thread is willing to pause and let other threads of the same or higher priority execute; however, it is not guaranteed to have any actual effect.

Q496. Is yield() guaranteed to pause the current thread?

A: No, yield() is merely a hint to the scheduler; the scheduler may ignore it entirely, and the same thread could continue executing immediately after calling yield().

Q497. What is the difference between sleep() and yield()?

A: sleep() guarantees the thread pauses for at least the specified duration, while yield() only suggests to the scheduler that the thread is willing to yield the CPU, with no guaranteed pause duration or effect.

65. WAIT()

Q498. What does wait() do?

A: wait() causes the current thread to release the object's monitor lock and pause until another thread calls notify() or notifyAll() on the same object, or a specified timeout elapses.

Q499. Where can wait() be called from?

A: wait() can only be called from within a synchronized context (synchronized method or block) on the object whose monitor is being waited on; otherwise, it throws IllegalMonitorStateException.

Q500. What is the difference between wait() and sleep()?

A: wait() releases the object's lock while the thread is paused and must be called within a synchronized block; sleep() does not release any locks and can be called from anywhere, without requiring synchronization.

Q501. What methods "wake up" a thread waiting due to wait()?

A: notify() (wakes up a single waiting thread) or notifyAll() (wakes up all waiting threads) called on the same object, from within a synchronized context.

Q502. Which class defines wait(), notify(), and notifyAll()?

A: They are defined in the Object class (not Thread), since every object in Java has an associated monitor that these methods operate on.

66. DIFFERENT STATES OF A THREAD

Q503. What are the different states of a thread in Java?

A: New, Runnable, Blocked, Waiting, Timed Waiting, and Terminated, as defined by the Thread.State enum.

Q504. What is the "New" state of a thread?

A: The state of a thread object that has been created but on which start() has not yet been called.

Q505. What is the "Runnable" state of a thread?

A: The state after start() has been called; the thread is either actively executing or ready/eligible to run and waiting for CPU allocation by the scheduler.

Q506. What is the "Blocked" state of a thread?

A: The state where a thread is waiting to acquire a monitor lock to enter a synchronized method/block that is currently held by another thread.

Q507. What is the "Waiting" state of a thread?

A: The state where a thread is waiting indefinitely for another thread to perform a specific action (e.g., after calling wait() without timeout, or join() without timeout).

Q508. What is the "Timed Waiting" state of a thread?

A: The state where a thread is waiting for a specified period of time (e.g., after calling sleep(millis), wait(millis), or join(millis)).

Q509. What is the "Terminated" state of a thread?

A: The state after a thread has completed execution of its run() method (either normally or due to an uncaught exception); a terminated thread cannot be restarted.

Q510. How do you check the current state of a thread?

A: Using the getState() method, which returns a value from the Thread.State enum.

67. COLLECTION VS COLLECTION FRAMEWORK

Q511. What is a Collection in Java?

A: A Collection is a single object/interface (java.util.Collection) that represents a group of individual objects treated as a single unit, with basic operations like add, remove, and iterate.

Q512. What is the Collection Framework?

A: The Collection Framework is a unified architecture (set of interfaces, implementing classes, and algorithms) in java.util package for storing and manipulating groups of objects efficiently, including List, Set, Queue, and Map hierarchies.

Q513. What is the difference between Collection and Collections?

A: Collection (singular) is an interface that represents a group of objects; Collections (plural) is a utility class containing static helper methods (like sort, reverse, synchronizedList) to operate on collection objects.

Q514. Is Map part of the Collection interface hierarchy?

A: No, the Map interface does not extend the Collection interface, though it is considered part of the broader Collection Framework since it works alongside collections and provides similar utility.

Q515. Why was the Collection Framework introduced?

A: To provide a standardized, reusable set of data structures and algorithms, replacing ad-hoc classes like Vector, Hashtable, and Array with a consistent, interoperable architecture.

68. OVERVIEW OF COLLECTION FRAMEWORK

Q516. What are the core interfaces of the Collection Framework?

A: Collection (root interface), List, Set, Queue (and Deque), and separately, Map (which doesn't extend Collection).

Q517. What is the List interface?

A: List is an ordered collection (sequence) that allows duplicate elements and provides index-based access; common implementations include ArrayList, LinkedList, and Vector.

Q518. What is the Set interface?

A: Set is a collection that does not allow duplicate elements; common implementations include HashSet, LinkedHashSet, and TreeSet.

Q519. What is the Queue interface?

A: Queue represents a collection designed for holding elements prior to processing, typically following FIFO (First-In-First-Out) order; implementations include LinkedList and PriorityQueue.

Q520. What is the Deque interface?

A: Deque (Double-Ended Queue) allows insertion and removal of elements from both ends; ArrayDeque and LinkedList are common implementations.

Q521. What is the difference between ArrayList and LinkedList?

A: ArrayList is backed by a dynamic array, offering fast random access (O(1)) but slower insertions/deletions in the middle (O(n)); LinkedList is backed by a doubly linked list, offering fast insertions/deletions (O(1)) but slower random access (O(n)).

Q522. What is the difference between HashSet, LinkedHashSet, and TreeSet?

A: HashSet offers no guaranteed order (uses hashing, O(1) average operations); LinkedHashSet maintains insertion order; TreeSet maintains elements in sorted order (using a Red-Black tree, O(log n) operations).

Q523. What is the difference between HashMap, LinkedHashMap, and TreeMap?

A: HashMap offers no guaranteed order; LinkedHashMap maintains insertion order; TreeMap maintains keys in sorted (natural or custom comparator) order.

Q524. What is the difference between ArrayList and Vector?

A: Both are backed by dynamic arrays with similar functionality, but Vector is synchronized (thread-safe, slower) while ArrayList is not synchronized (faster, but not thread-safe by default).

Q525. What is the difference between HashMap and Hashtable?

A: HashMap allows one null key and multiple null values and is not synchronized; Hashtable does not allow any null keys or values and is synchronized (thread-safe but slower and considered legacy).

Q526. What is a PriorityQueue?

A: A Queue implementation where elements are ordered according to their natural ordering or a provided Comparator, with the smallest (or highest priority) element always at the head for retrieval.

Q527. Does Set maintain insertion order?

A: It depends on implementation: HashSet does not guarantee order, LinkedHashSet maintains insertion order, and TreeSet maintains sorted order (not insertion order).

Q528. What is the initial capacity and load factor of a HashMap?

A: Default initial capacity is 16, and default load factor is 0.75, meaning the HashMap resizes (doubles capacity) once it is 75% full.

69. CURSORS IN JAVA

Q529. What are cursors in Java Collections?

A: Cursors are mechanisms used to traverse (iterate over) elements of a collection one by one; Java provides three types: Enumeration, Iterator, and ListIterator.

Q530. What is Enumeration in Java?

A: Enumeration is a legacy cursor interface (used with older classes like Vector and Hashtable) that allows forward-only traversal of a collection, but does not support element removal during iteration.

Q531. What is Iterator in Java?

A: Iterator is a universal cursor that works with any Collection implementation, allowing forward-only traversal, and supports element removal from the collection during iteration via its remove() method.

Q532. What is ListIterator in Java?

A: ListIterator is a specialized cursor available only for List implementations, supporting bidirectional traversal (forward and backward), and allowing add, remove, and set (replace) operations during iteration.

Q533. What are the key methods of Iterator?

A: hasNext() (checks if more elements exist), next() (returns the next element), and remove() (removes the last element returned by next()).

Q534. What are the key methods of ListIterator (in addition to Iterator's methods)?

A: hasPrevious(), previous(), nextIndex(), previousIndex(), add(), and set().

Q535. What is the difference between Iterator and ListIterator?

A: Iterator supports only forward traversal and works on all Collection types; ListIterator supports both forward and backward traversal but works only on List implementations, and additionally supports add() and set() operations.

Q536. What is ConcurrentModificationException?

A: A runtime exception thrown when a collection is structurally modified (e.g., adding/removing elements) while being iterated using an Iterator or for-each loop, other than through the iterator's own remove()/add() methods.

Q537. How can you safely remove elements from a collection while iterating?

A: By using the Iterator's own remove() method (iterator.remove()) instead of directly calling the collection's remove() method during iteration, which avoids ConcurrentModificationException.

70. COMPARABLE VS COMPARATOR

Q538. What is the Comparable interface?

A: Comparable is an interface with a single method, compareTo(Object o), implemented by a class to define its natural (default) ordering, allowing its objects to be sorted directly (e.g., using Collections.sort()).

Q539. What is the Comparator interface?

A: Comparator is a functional interface with a compare(Object o1, Object o2) method, used to define custom/external ordering logic for objects, separate from the class itself, allowing multiple different sorting sequences.

Q540. What is the key difference between Comparable and Comparator?

A: Comparable is implemented by the class itself to define a single natural ordering (compareTo modifies the class), while Comparator is a separate class/lambda used to define one or more custom orderings without modifying the original class.

Q541. What does compareTo() return?

A: A negative integer if the current object is less than the other, zero if equal, and a positive integer if greater than the other object, per Java convention.

Q542. What package do Comparable and Comparator belong to?

A: Comparable is in java.lang; Comparator is in java.util.

Q543. How do you sort a list using a custom Comparator?

A: By passing a Comparator implementation (or lambda expression) to Collections.sort(list, comparator) or list.sort(comparator).

Q544. Can a class implement multiple Comparable interfaces for different sort criteria?

A: No, a class can only implement Comparable once, providing a single natural ordering; for multiple different sort orders, separate Comparator implementations must be created instead.

Q545. What is the use of Comparator.comparing() (Java 8+)?

A: It is a static factory method used to create a Comparator concisely based on a key extraction function, often combined with thenComparing() for multi-level sorting.

71. AUTOBOXING AND UNBOXING

Q546. What is autoboxing in Java?

A: Autoboxing is the automatic conversion performed by the Java compiler between a primitive type and its corresponding wrapper class object (e.g., int to Integer), done implicitly.

Q547. What is unboxing in Java?

A: Unboxing is the reverse process—the automatic conversion of a wrapper class object back to its corresponding primitive type (e.g., Integer to int), done implicitly by the compiler.

Q548. When was autoboxing/unboxing introduced in Java?

A: Java 5 (J2SE 5.0).

Q549. Why is autoboxing/unboxing useful, especially with collections?

A: Because Java collections (like ArrayList) can only store objects, not primitives; autoboxing allows primitives to be seamlessly added to collections, and unboxing allows them to be retrieved and used as primitives again.

Q550. What are wrapper classes in Java?

A: Wrapper classes provide an object representation for each primitive data type: Integer, Long, Short, Byte, Float, Double, Character, and Boolean.

Q551. Can autoboxing cause a NullPointerException?

A: Yes, if a wrapper object holding a null value is unboxed (e.g., Integer i = null; int x = i;), it throws a NullPointerException at runtime during the unboxing attempt.

Q552. What is the Integer cache in Java (related to autoboxing)?

A: Java caches Integer objects for values from -128 to 127; autoboxing within this range reuses cached objects (so == comparison may return true), while values outside this range create new objects each time (so == may return false).

Q553. Is autoboxing/unboxing performance-neutral compared to using primitives directly?

A: No, autoboxing/unboxing incurs some performance overhead (object creation, unwrapping) compared to using primitives directly, which matters in performance-critical code, especially inside loops.

72. GENERICS

Q554. What are generics in Java?

A: Generics allow classes, interfaces, and methods to operate on types specified as parameters, enabling type safety at compile time and eliminating the need for explicit casting.

Q555. Why were generics introduced in Java?

A: To provide compile-time type checking (catching type errors early) and eliminate the need for manual casting when retrieving elements from collections, which was error-prone in pre-Java 5 code.

Q556. What is type erasure in generics?

A: Type erasure is the process by which the compiler removes all generic type information after compile-time checks, replacing type parameters with their bounds (or Object) in the compiled bytecode, since generics don't exist at runtime.

Q557. What is a generic class? Give an example.

A: A class defined with one or more type parameters, e.g., class Box<T> { private T value; ... }, allowing the same class to work with different data types safely.

Q558. What is a bounded type parameter in generics?

A: A type parameter restricted to a specific range of types using 'extends', e.g., <T extends Number>, meaning T must be Number or one of its subclasses.

Q559. What are wildcards in generics?

A: The '?' symbol representing an unknown type, used in generic method parameters, e.g., List<?> for any type, List<? extends Number> for upper-bounded, and List<? super Integer> for lower-bounded wildcards.

Q560. Can you create a generic array in Java (e.g., new T[])?

A: No, due to type erasure, creating a generic array directly (like new T[10]) is not allowed and causes a compile-time error; workarounds involve creating an Object[] and casting.

Q561. Can primitive types be used as generic type parameters?

A: No, generics only work with reference (object) types; primitives must be used via their wrapper classes (e.g., List<Integer> instead of List<int>).

Q562. What is a generic method?

A: A method that introduces its own type parameter(s), independent of the class it belongs to, e.g., public <T> void print(T item) { }.

73. FAIL-FAST VS FAIL-SAFE

Q563. What are fail-fast iterators?

A: Fail-fast iterators immediately throw a ConcurrentModificationException if the underlying collection is structurally modified after the iterator is created (except via the iterator's own methods), such as those from ArrayList and HashMap.

Q564. What are fail-safe iterators?

A: Fail-safe iterators do not throw an exception if the underlying collection is modified during iteration; they typically operate on a cloned/snapshot copy of the collection, such as those from CopyOnWriteArrayList and ConcurrentHashMap.

Q565. How do fail-fast iterators detect concurrent modification?

A: They use an internal counter called modCount, which is incremented on every structural modification; the iterator checks if modCount has changed unexpectedly during iteration and throws ConcurrentModificationException if so.

Q566. What is a downside of fail-safe iterators?

A: Since they may operate on a copy/snapshot of the data, they might not reflect the most recent updates made to the collection during iteration, and creating copies can have memory/performance overhead.

Q567. Give examples of fail-fast and fail-safe collection classes.

A: Fail-fast: ArrayList, HashMap, HashSet. Fail-safe: CopyOnWriteArrayList, ConcurrentHashMap, CopyOnWriteArraySet.

74. COLLECTIONS UTILITY CLASS

Q568. What is the Collections class in Java?

A: Collections (java.util.Collections) is a utility class containing only static methods that operate on or return collections, such as sorting, searching, and synchronization helpers.

Q569. What are common methods of the Collections utility class?

A: sort(), reverse(), shuffle(), max(), min(), binarySearch(), unmodifiableList(), synchronizedList(), and emptyList().

Q570. What does Collections.unmodifiableList() do?

A: It returns a read-only (immutable) view of the specified list; any attempt to modify it throws UnsupportedOperationException.

Q571. What does Collections.synchronizedList() do?

A: It returns a thread-safe (synchronized) wrapper around the specified list, ensuring that access to it is synchronized, though manual synchronization is still needed when iterating.

Q572. How do you sort a List using the Collections class?

A: Using Collections.sort(list) for natural ordering (requires elements to implement Comparable), or Collections.sort(list, comparator) for custom ordering.

Q573. What does Collections.emptyList() return?

A: It returns an immutable, empty List instance, useful for returning an empty result without allocating a new list each time.

75. ARRAYS UTILITY CLASS

Q574. What is the Arrays class in Java?

A: Arrays (java.util.Arrays) is a utility class containing static methods for performing operations on arrays, such as sorting, searching, filling, and converting to other collection types.

Q575. What are common methods of the Arrays utility class?

A: sort(), binarySearch(), equals(), fill(), copyOf(), copyOfRange(), asList(), toString(), and deepToString().

Q576. What does Arrays.asList() do?

A: It converts an array into a fixed-size List view backed by the original array; the returned list's size cannot be changed (add/remove throws UnsupportedOperationException), though existing elements can be modified.

Q577. What is the difference between Arrays.sort() for primitives versus objects?

A: For primitive arrays, Arrays.sort() uses a dual-pivot quicksort algorithm; for object arrays, it uses a modified mergesort/Timsort algorithm, which is stable (preserves relative order of equal elements).

Q578. What does Arrays.fill() do?

A: It assigns the specified value to every element (or a specified range) of an array.

Q579. What does Arrays.equals() check?

A: It checks whether two arrays are structurally equal—same length and all corresponding elements equal—returning a boolean.

Q580. What is the difference between toString() and deepToString() in the Arrays class?

A: toString() provides a string representation of a one-dimensional array's elements; deepToString() is used for multi-dimensional (nested) arrays, recursively converting nested array elements into a readable string form.

76. OVERVIEW OF MAPS

Q581. What is a Map in Java?

A: A Map is an object that maps unique keys to values, where each key can map to at most one value; it does not extend the Collection interface but is part of the Collection Framework.

Q582. What are the main implementations of the Map interface?

A: HashMap, LinkedHashMap, TreeMap, and Hashtable.

Q583. Can a Map contain duplicate keys?

A: No, a Map cannot have duplicate keys—each key is unique; however, multiple keys can map to the same (duplicate) value.

Q584. Does HashMap allow null keys and values?

A: Yes, HashMap allows one null key and multiple null values.

Q585. Does TreeMap allow null keys?

A: No, TreeMap does not allow null keys (throws NullPointerException) since it needs to compare keys for sorting, though it can allow null values.

Q586. What is the difference between HashMap and TreeMap in terms of ordering?

A: HashMap does not maintain any specific order of keys; TreeMap maintains keys in sorted order (natural ordering or a custom Comparator).

Q587. What are common methods of the Map interface?

A: put(), get(), remove(), containsKey(), containsValue(), keySet(), values(), entrySet(), size(), and isEmpty().

Q588. What does entrySet() return?

A: It returns a Set view of the key-value mappings (as Map.Entry objects) contained in the map, commonly used to iterate over both keys and values together.

Q589. What is Map.Entry?

A: Map.Entry is a nested interface within Map representing a single key-value pair, providing getKey() and getValue() methods to access each part.

Q590. How does HashMap handle collisions (two keys with the same hash bucket)?

A: HashMap uses chaining—colliding entries are stored in a linked list within the same bucket (or, since Java 8, converted to a balanced tree/Red-Black tree if the bucket grows beyond a threshold, for better worst-case performance).

Q591. What is the role of hashCode() and equals() in HashMap?

A: hashCode() determines which bucket a key-value pair is placed in, while equals() is used to check for key equality within that bucket (to detect duplicates or retrieve the correct value); both must be consistently overridden together for custom key objects.

Q592. What is LinkedHashMap used for?

A: LinkedHashMap extends HashMap and maintains a doubly-linked list running through its entries, preserving insertion order (or optionally access order) when iterating.

Q593. Is HashMap thread-safe?

A: No, HashMap is not synchronized/thread-safe by default; ConcurrentHashMap or Collections.synchronizedMap() should be used for thread-safe access in multithreaded environments.

Q594. What is ConcurrentHashMap?

A: A thread-safe implementation of the Map interface designed for high-concurrency scenarios, using internal lock-striping (segmenting) to allow multiple threads to read/write concurrently without locking the entire map.

Q595. What happens if you put a key that already exists into a HashMap?

A: The existing value associated with that key is replaced/overwritten with the new value, and the old value is returned by the put() method.

Q596. What is the time complexity of get() and put() operations in HashMap?

A: On average, O(1) (constant time), assuming a good hash distribution; in the worst case (many collisions), it can degrade to O(n) or O(log n) with Java 8's treeification of buckets.

Q597. How do you iterate over a Map in Java?

A: Commonly by calling entrySet() and iterating with a for-each loop to access both key and value together, or by iterating over keySet() and calling get(key) for each, or using forEach() with a lambda (Java 8+).

Q598. What is the difference between keySet(), values(), and entrySet()?

A: keySet() returns a Set view of all keys; values() returns a Collection view of all values; entrySet() returns a Set view of all key-value pairs (Map.Entry objects).

Q599. What is a NavigableMap and how does TreeMap use it?

A: NavigableMap extends SortedMap, adding navigation methods like floorKey(), ceilingKey(), higherKey(), and lowerKey(); TreeMap implements NavigableMap, providing these methods for finding closest matches around a given key.

Q600. What is the default capacity and resizing behavior of a HashMap?

A: The default initial capacity is 16 buckets with a load factor of 0.75; when the number of entries exceeds capacity × load factor, the HashMap resizes by doubling its capacity and rehashing all entries.

FOUR PILLARS OF OOP

Q601. What are the four pillars of Object-Oriented Programming? A: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Q602. What is encapsulation in Java? A: Encapsulation is the practice of bundling data (fields) and the methods that operate on that data within a single class, while restricting direct access to the internal state using access modifiers (typically private fields with public getters/setters).

Q603. What is abstraction in Java? A: Abstraction is the concept of hiding implementation details and exposing only the essential features/behavior of an object, achieved in Java through abstract classes and interfaces.

Q604. What is the difference between abstraction and encapsulation? A: Abstraction focuses on hiding complexity by showing only relevant behavior (design-level concern, "what" an object does), while encapsulation focuses on hiding internal data by restricting access (implementation-level concern, "how" it's protected).

Q605. How does Java achieve polymorphism? A: Through method overloading (compile-time polymorphism) and method overriding (runtime polymorphism).

Q606. What is a POJO in Java? A: POJO (Plain Old Java Object) is a simple Java object that does not extend or implement any special framework classes/interfaces, typically containing private fields with public getters/setters, used to represent data.

Q607. What is a JavaBean? A: A JavaBean is a POJO that follows specific conventions: a public no-arg constructor, private fields accessed via public getter/setter methods following naming conventions, and implementing Serializable.

78. PACKAGES

Q608. What is a package in Java? A: A package is a namespace/grouping mechanism used to organize related classes and interfaces together, helping avoid naming conflicts and controlling access.

Q609. What are the types of packages in Java? A: Built-in (predefined) packages like java.util and java.io, and user-defined packages created by programmers.

Q610. How do you create and use a package? A: Declare 'package packageName;' as the first statement in the file, and use 'import packageName.ClassName;' (or packageName.*) in other files to access its classes.

Q611. What is the default package? A: If a class does not declare a package, it belongs to the unnamed/default package; classes in the default package cannot be imported by classes in named packages.

Q612. What is static import? A: Static import (import static packageName.ClassName.member;) allows static members of a class to be used directly without qualifying them with the class name, e.g., importing Math.PI to use PI directly. **Q613. Does the directory structure need to match the package name?** A: Yes, the compiled .class file must reside in a directory structure that mirrors the package declaration (e.g., package com.app; requires the file in a com/app folder).

79. ENUMS

Q614. What is an enum in Java? A: An enum is a special data type (declared with the 'enum' keyword) that represents a fixed set of named constants, implicitly extending java.lang.Enum.

Q615. Can an enum have constructors, fields, and methods? A: Yes, an enum can have private constructors (implicitly private), fields, and both instance and abstract methods, allowing each constant to optionally override behavior.

Q616. Can an enum implement an interface? A: Yes, an enum can implement one or more interfaces, though it cannot extend any class since it already implicitly extends Enum.

Q617. What are common methods available on enums? A: values() (returns an array of all constants), valueOf(String) (returns the constant matching the given name), ordinal() (returns the position/index), and name() (returns the constant's name as declared).

Q618. Can enum constants be used in a switch statement? A: Yes, enums are commonly used as switch expressions/statements, referring to the constant names directly without qualifying them with the enum type.

Q619. Is an enum type-safe compared to using int constants? A: Yes, enums provide compile-time type safety since only valid predefined constants of that enum type can be assigned, unlike int constants which allow any arbitrary integer value.

80. NESTED AND INNER CLASSES

Q620. What is a nested class in Java? A: A class defined within another class; nested classes are categorized as static nested classes and inner (non-static) classes.

Q621. What is a static nested class? A: A nested class declared with the static keyword; it does not hold an implicit reference to an instance of the outer class and can be instantiated independently using OuterClass.NestedClass.

Q622. What is an inner (non-static) class? A: A non-static nested class that holds an implicit reference to an instance of its enclosing outer class, and requires an outer class instance to be created (outer.new Inner()).

Q623. What is a local inner class? A: A class defined within a method body; its scope is limited to that method, and it can access effectively final local variables of the enclosing method.

Q624. What is an anonymous inner class? A: A class without a name, declared and instantiated simultaneously, typically used to provide a one-time implementation of an interface or abstract/concrete class.

Q625. Why would you use an anonymous inner class? A: To create a short-lived, one-off implementation of an interface or class (e.g., an event listener or Runnable) without formally declaring a separate named class.

Q626. Can a static nested class access non-static members of the outer class? A: No, a static nested class cannot directly access instance (non-static) members of the outer class, since it isn't tied to any outer instance; it would need an explicit outer object reference.

Q627. How does an inner class access the outer class's members if there's a naming conflict? A: Using OuterClassName.this.memberName to explicitly refer to the outer instance's member instead of the inner class's own.

81. LAMBDA EXPRESSIONS AND FUNCTIONAL INTERFACES

Q628. What is a lambda expression in Java? A: A lambda expression (introduced in Java 8) is a concise way to represent an anonymous function, written as (parameters) -> expression/body, used primarily to implement functional interfaces.

Q629. What is required for a lambda expression to be used? A: The target type must be a functional interface (an interface with exactly one abstract method).

Q630. What are common built-in functional interfaces in java.util.function? A: Function<T,R>, Predicate<T>, Supplier<T>, Consumer<T>, BiFunction<T,U,R>, and UnaryOperator<T>.

Q631. What is a method reference in Java? A: A method reference (using the :: syntax) is a shorthand for a lambda expression that simply calls an existing method, e.g., System.out::println instead of x -> System.out.println(x).

Q632. What are the types of method references? A: Reference to a static method (ClassName::staticMethod), reference to an instance method of a particular object

(obj::instanceMethod), reference to an instance method of an arbitrary object of a type (ClassName::instanceMethod), and reference to a constructor (ClassName::new).

Q633. Can a lambda expression access variables from its enclosing scope? A: Yes, but only variables that are final or effectively final (never reassigned after initialization); attempting to use a mutable local variable causes a compile-time error.

Q634. Why must variables captured by a lambda be effectively final? A: Because lambdas may be executed later or on a different thread, and capturing a mutable variable could lead to inconsistent or unpredictable behavior; restricting to effectively final variables avoids this.

82. STREAM API

Q635. What is the Stream API in Java? A: Introduced in Java 8, the Stream API (java.util.stream) provides a functional approach to process sequences of elements (from collections, arrays, etc.) through operations like filter, map, and reduce, without modifying the original source.

Q636. How do you create a Stream from a Collection? A: Using the stream() method available on Collection implementations, e.g., list.stream();

Q637. What is the difference between intermediate and terminal operations in streams? A: Intermediate operations (like filter, map, sorted) are lazy and return a new Stream, allowing chaining; terminal operations (like collect, forEach, reduce) trigger actual processing and produce a result or side effect, after which the stream cannot be reused.

Q638. What is the difference between map() and filter() in streams? A: map() transforms each element into another form/value (one-to-one transformation); filter() selects elements matching a given predicate, potentially reducing the number of elements without transforming them.

Q639. What does the reduce() method do in streams? A: It combines all elements of the stream into a single result by repeatedly applying a binary operator (e.g., summing all elements).

Q640. What is the collect() method used for? A: It is a terminal operation that accumulates stream elements into a different form, commonly a Collection (List, Set) or Map, using a Collector (e.g., Collectors.toList()).

Q641. Can a Stream be reused after a terminal operation is called? A: No, once a terminal operation is invoked, the stream is considered consumed/closed; attempting to reuse it throws IllegalStateException.

Q642. What is the difference between a sequential and parallel stream? A: A sequential stream processes elements one at a time in a single thread; a parallel stream (created via parallelStream() or stream().parallel()) splits the work across multiple threads for potentially faster processing on multi-core systems.

83. OPTIONAL CLASS

Q643. What is the Optional class in Java? A: Optional<T> (introduced in Java 8) is a container object that may or may not hold a non-null value, used to represent the potential absence of a value and reduce explicit null checks/NullPointerExceptions.

Q644. How do you create an Optional? A: Using Optional.of(value) (throws NPE if value is null), Optional.ofNullable(value) (allows null, resulting in an empty Optional), or Optional.empty() (an explicitly empty Optional).

Q645. What are common methods of the Optional class? A: isPresent(), isEmpty(), get(), orElse(defaultValue), orElseGet(supplier), orElseThrow(), ifPresent(consumer), and map().

Q646. What is the risk of calling get() on an Optional without checking isPresent()? A: If the Optional is empty, get() throws NoSuchElementException; it's generally safer to use orElse(), orElseGet(), or ifPresent() instead.

84. OBJECT CLASS METHODS

Q647. What methods does every Java class inherit from Object? A: equals(), hashCode(), toString(), getClass(), clone(), wait(), notify(), notifyAll(), and finalize().

Q648. What is the default implementation of equals() in the Object class? A: It performs a reference comparison (==), returning true only if both references point to the exact same object in memory.

Q649. Why should equals() and hashCode() be overridden together? A: Because of the hashCode-equals contract: if two objects are equal per equals(), they must have the same hashCode(); violating this breaks the correct behavior of hash-based collections like HashMap and HashSet.

Q650. What is the contract of the equals() method? A: It must be reflexive (x.equals(x) is true), symmetric (x.equals(y) implies y.equals(x)), transitive (if x.equals(y) and y.equals(z), then x.equals(z)), consistent (repeated calls return the same result if objects are unchanged), and x.equals(null) must return false.

Q651. What does the default toString() method return? A: It returns a string in the format getClass().getName() + "@" + Integer.toHexString(hashCode()), unless overridden.

Q652. What does getClass() return? A: It returns the runtime Class object representing the actual class of the object, useful for reflection.

Q653. What is the difference between shallow cloning and deep cloning? A: Shallow cloning (default clone()) copies an object's fields as-is, so reference fields still point to the same nested objects; deep cloning creates entirely independent copies of any nested/referenced objects as well.

Q654. What must a class do to support cloning via clone()? A: It must implement the Cloneable marker interface; otherwise, calling clone() throws CloneNotSupportedException.

Q655. What is a marker interface? A: An interface with no methods or fields (empty body) used purely to "mark" a class as having a certain capability or property, which the JVM/libraries check via instanceof, e.g., Serializable, Cloneable, Remote.

Q656. What is the finalize() method? A: A method that was historically called by the garbage collector before reclaiming an object's memory, intended for cleanup; it is deprecated (since Java 9) and unreliable, with try-with-resources/AutoCloseable now preferred.

85. SERIALIZATION

Q657. What is serialization in Java? A: Serialization is the process of converting an object's state into a byte stream, so it can be saved to a file, sent over a network, or stored, and later reconstructed via deserialization.

Q658. How do you make a class serializable? A: By implementing the Serializable marker interface (which has no methods), signaling to the JVM that instances of the class can be converted to a byte stream.

Q659. What is serialVersionUID? A: A unique identifier (a long value) associated with a Serializable class, used during deserialization to verify that the sender and receiver of a serialized object have compatible class versions; a mismatch throws InvalidClassException.

Q660. What happens to static and transient fields during serialization? A: Static fields are not serialized since they belong to the class, not the object instance; fields marked transient are also excluded from serialization and are restored to their default values upon deserialization.

Q661. What happens if a class implementing Serializable has a non-serializable field? A: A NotSerializableException is thrown at runtime during serialization, unless that field is marked transient or the field's class also implements Serializable.

Q662. What is the difference between Serializable and Externalizable? A: Serializable uses default JVM serialization logic automatically; Externalizable requires the class to implement writeExternal() and readExternal() methods, giving the programmer full manual control over the serialization format.

86. VARARGS

Q663. What are varargs in Java? A: Varargs (variable arguments), denoted by an ellipsis (...) after the type, allow a method to accept a variable number of arguments of the specified type, treated internally as an array.

Q664. What are the rules for using varargs in a method signature? A: A method can have only one varargs parameter, and it must be the last parameter in the parameter list.

Q665. Can varargs be combined with method overloading, and how is ambiguity resolved? A: Yes, but if an exact-match (non-varargs) overloaded method exists, Java prefers it over the varargs version; varargs is used as a last resort during overload resolution.

Q666. How are varargs treated internally by the compiler? A: The compiler treats varargs as an array, so within the method body, the varargs parameter behaves exactly like an array of that type.

87. SWITCH STATEMENT AND LOOPS

Q667. What data types can be used in a switch statement/expression? A: byte, short, char, int (and their wrapper classes), String (since Java 7), and enum types.

Q668. What is fall-through behavior in a switch statement? A: Without a break statement, execution continues into subsequent case blocks even if their condition doesn't match, until a break is encountered or the switch ends.

Q669. What is a switch expression (Java 14+)? A: A newer form of switch that can be used as an expression (returning a value) using the arrow syntax (case X -> value;), which does not fall through and requires the yield keyword for multi-statement blocks.

Q670. What is a labeled loop in Java? A: A loop prefixed with an identifier (label:) that allows break or continue statements to target that specific outer loop directly, useful for exiting/continuing nested loops.

. What is the difference between break and continue? A: break terminates the loop (or switch) entirely and transfers control to the statement after it; continue skips the rest of the current iteration and proceeds to the next iteration of the loop.

Q672. What is an enhanced for loop (for-each)? A: A simplified loop syntax (for (Type var : collection/array)) used to iterate over all elements of an array or Iterable without needing an explicit index or iterator.

Q673. Can you modify a collection using a for-each loop directly? A: No, structurally modifying (adding/removing elements) a collection directly within a for-each loop throws ConcurrentModificationException; an explicit Iterator's remove() method must be used instead.

88. GARBAGE COLLECTION

Q674. What is garbage collection in Java? A: Garbage collection is the automatic process by which the JVM identifies and reclaims memory occupied by objects that are no longer reachable/referenced by the running program.

Q675. How does the JVM determine an object is eligible for garbage collection? A: An object becomes eligible when it is no longer reachable through any live reference chain from active threads, static fields, or JVM roots.

Q676. Can a programmer force garbage collection to run? A: Not directly; calling System.gc() or Runtime.gc() only suggests/requests garbage collection to the JVM, but it's not guaranteed to run immediately or at all.

Q677. What are the generations in the JVM heap used for garbage collection? A: The heap is typically divided into the Young Generation (further split into Eden and Survivor spaces, for newly created objects) and the Old (Tenured) Generation (for long-lived objects that survive multiple garbage collection cycles).

Q678. What is a minor GC versus a major/full GC? A: A minor GC cleans the Young Generation only (fast, frequent); a major/full GC cleans the Old Generation (and often the entire heap), which is slower and less frequent.

Q679. How does an object move from the Young Generation to the Old Generation? A: If an object survives multiple minor garbage collection cycles in the Young Generation (surviving repeated Eden/Survivor sweeps), it gets promoted ("tenured") to the Old Generation.

89. IMMUTABLE CLASSES

Q680. What is an immutable class? A: A class whose instances cannot be modified after creation; once an object is constructed, its state remains constant throughout its lifetime.

Q681. How do you create an immutable class in Java? A: Declare the class as final (prevent subclassing), make all fields private and final, don't provide setters, initialize all fields via the constructor, and for mutable field types, return defensive copies from getters instead of direct references.

Q682. Give examples of immutable classes in the Java standard library. A: String, Integer and other wrapper classes, and LocalDate/LocalDateTime (from java.time).

Q683. What are the benefits of immutability? A: Thread-safety without synchronization, safe sharing/caching of instances, simpler reasoning about state, and protection against unintended side effects.

90. VAR, RECORDS, AND OTHER MODERN FEATURES

Q684. What is the 'var' keyword in Java (Java 10+)? A: 'var' enables local variable type inference, allowing the compiler to automatically determine a local variable's type from the assigned value, e.g., var list = new ArrayList<String>();

Q685. Can 'var' be used for instance variables or method parameters? A: No, 'var' can only be used for local variables with an initializer, not for instance/static fields, method parameters, or return types.

Q686. What is a record in Java (Java 16+)? A: A record is a concise way to declare an immutable data-carrier class; the compiler automatically generates a constructor, accessor methods, equals(), hashCode(), and toString() based on the declared components.

Q687. What is a text block in Java (Java 15+)? A: A text block is a multi-line string literal delimited by triple double-quotes (""") that preserves formatting and reduces the need for escape characters and concatenation.

Q688. What are sealed classes/interfaces (Java 17+)? A: Sealed classes/interfaces restrict which other classes or interfaces may extend or implement them, explicitly listed using the 'permits' clause, providing more control over class hierarchies.

91. REFLECTION

Q689. What is reflection in Java? A: Reflection (java.lang.reflect) is an API that allows a program to inspect and manipulate classes, methods, fields, and constructors at runtime, even those it doesn't have compile-time knowledge of.

Q690. How do you obtain a Class object for reflection? A: Using ClassName.class, object.getClass(), or Class.forName("fully.qualified.ClassName").

Q691. What can reflection be used for? A: Inspecting class metadata, dynamically invoking methods/constructors, accessing (and even modifying) private fields, and building frameworks like dependency injection or ORM libraries.

Q692. What are the downsides of using reflection? A: Performance overhead, breaks encapsulation (can bypass access modifiers), reduced compile-time type safety, and increased code complexity/fragility.

92. ANNOTATIONS

Q693. What is an annotation in Java? A: An annotation is a form of metadata (prefixed with @) added to code elements (classes, methods, fields) that provides information to the compiler or runtime tools without directly affecting program logic.

Q694. What are common built-in annotations in Java? A: @Override, @Deprecated, @SuppressWarnings, @FunctionalInterface, and @SafeVarargs.

Q695. What does @Deprecated indicate? A: It marks a method, class, or field as outdated and discouraged for use, typically generating a compiler warning when referenced, often with guidance toward a replacement.

Q696. What does @SuppressWarnings do? A: It instructs the compiler to suppress specific categories of warnings (e.g., "unchecked", "deprecation") for the annotated element.

Q697. Can custom annotations be created in Java? A: Yes, using the @interface keyword to define a custom annotation, optionally specifying retention policy (@Retention) and target elements (@Target) it applies to.

698. What is the difference between "is-a" and "has-a" relationships in OOP? A: "is-a" represents inheritance (a subclass is a type of its superclass, e.g., Dog is-a Animal), while "has-a" represents composition/aggregation (a class contains a reference to another class as a field, e.g., Car has-a Engine).

Q699. Is Java a 100% pure object-oriented language? A: No, Java is not purely object-oriented because it supports primitive data types (int, char, boolean, etc.) that are not objects, unlike languages such as Smalltalk where everything is an object.

Q700. Can you give a real-world example illustrating all four OOP pillars together? A: A BankAccount class illustrates all four: encapsulation (balance kept private with deposit/withdraw methods), abstraction (users call withdraw() without knowing the internal validation logic), inheritance (SavingsAccount and CurrentAccount extend BankAccount), and polymorphism (each subclass overrides calculateInterest() differently while being referenced as a common BankAccount type).